WARWICK
THE UNIVERSITY OF WARWICK

Lecture 10
Grammars and Parsing

Gabriele Pergola

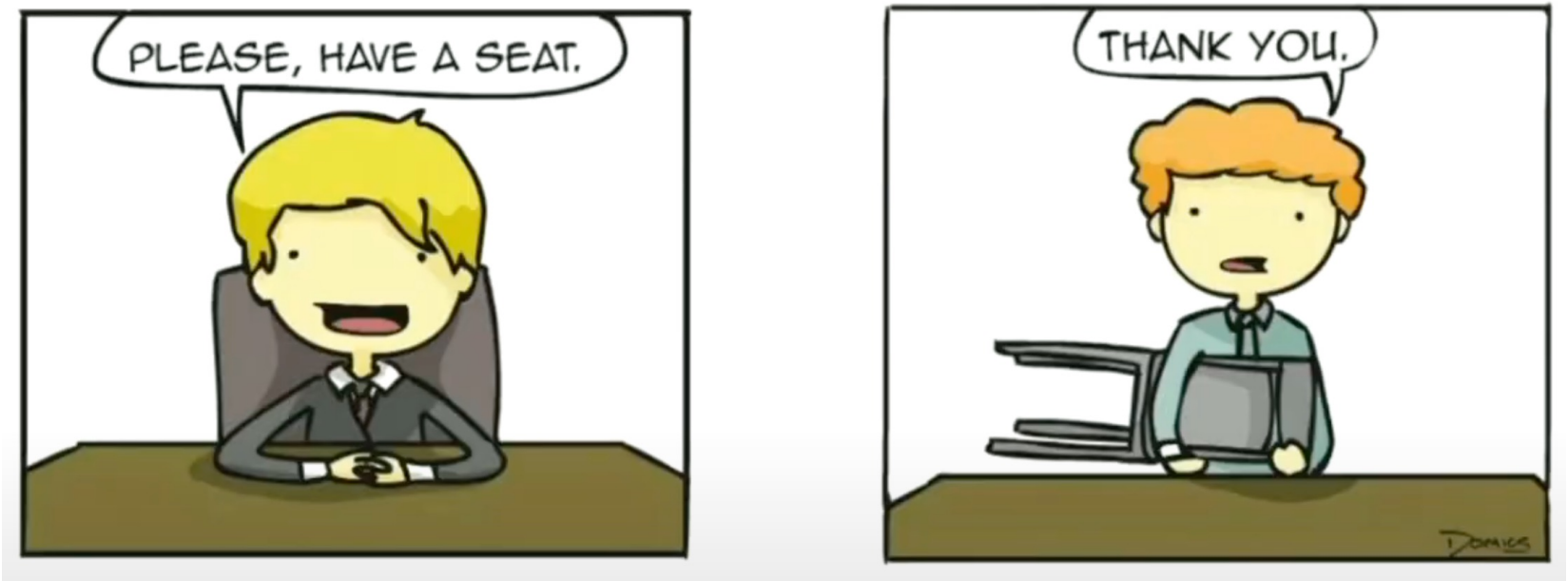
Natural Language Processing
CS918

February 2026

Outline

- How can we interpret and compute the meaning of a sentence?
- Introduce the concept of **syntax** and **parsing**
- What are **constituencies** and **dependency structures**?
- **Probabilistic parsing**: Context Free Grammars (CFG and PCFG)
  - Lexicalised parsing
- **Dependency parsing**

Meaning?



Credits to: Meaning Representation for Natural Language Understanding - Mariana Romanyshyn

How do we understand the meaning of a sentence?

- we know what words mean
- we know how words interconnect
- we know the context of the sentence
- we know a lot about the world around us
- we use non-verbal communication

HOW CAN WE INTERPRET AND 'COMPUTE' THE MEANING OF A SENTENCE?

Option 1: Bag-of-Words

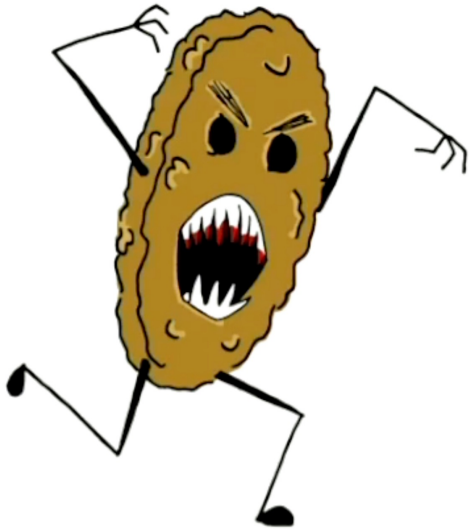
- **Text:** Children wanted to eat all the cookies. They were delicious!

Option 1: Bag-of-Words

- **Text:** Children wanted to eat all the cookies. They were delicious!
- **Meaning:** cookie, child, eat, delicious

Option 1: Bag-of-Words

- **Text:** Cookies wanted to eat all the children. They were delicious!



Credits to: Meaning Representation for Natural Language Understanding - Mariana Romanyshyn

Option 1: Bag-of-Words

- **Text1 :** Children wanted to eat all the cookies. They were delicious!
- **Meaning:** cookie, child, eat, delicious
- **Text 2:** Cookies wanted to eat all the children. They were delicious!
- **Meaning:** cookie, child, eat, delicious

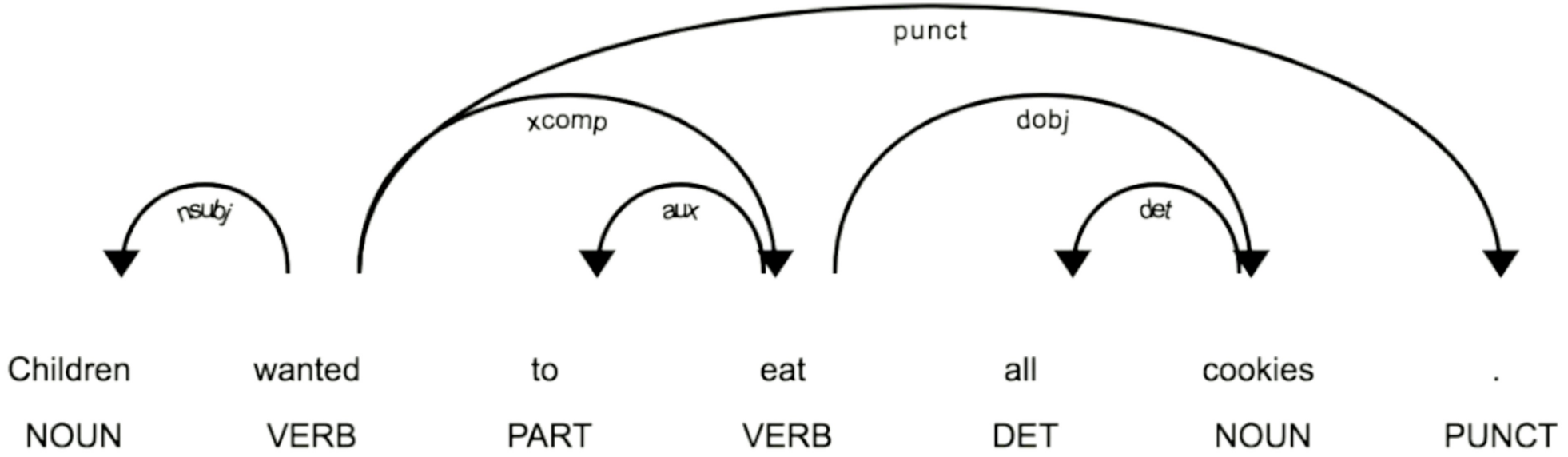
Let’s introduce syntax!

Children wanted to eat all cookies .

Let’s introduce syntax!

Children	wanted	to	eat	all	cookies	.
NOUN	VERB	PART	VERB	DET	NOUN	PUNCT

Let’s introduce syntax!



Credits to: Meaning Representation for Natural Language Understanding - Mariana Romanyshyn

Why is syntax important?

- Parsing is key for applications needing **deep understanding of language**, e.g.:
  - Grammar checkers
  - Information extraction
  - Machine translation
  - Dialogue management
  - Question answering

Let's introduce syntax … which means: **parsing!**

- **Parsing**: process of recognising a sentence and assigning **syntactic structure** to it.

- Syntactic structure includes:
  - **Constituents**: how words group together to behave as single units.
  - **Grammatical relations, dependencies, subcategorisation**: how words and constituents relate to each other.

Two types of syntactical structures

1. **Constituency** structures
2. **Dependency** structures

Two types of syntactical structures

1. **Constituency** structures
2. **Dependency** structures

1. Constituency structures

A sentence is organized into **constituencies:** hierarchical and nested units.

E.g.: " the cuddly cat by the door "

Starting unit - words:

the, cat, cuddly, by, door

DET N ADJ PREP N

Words combine into phrases:

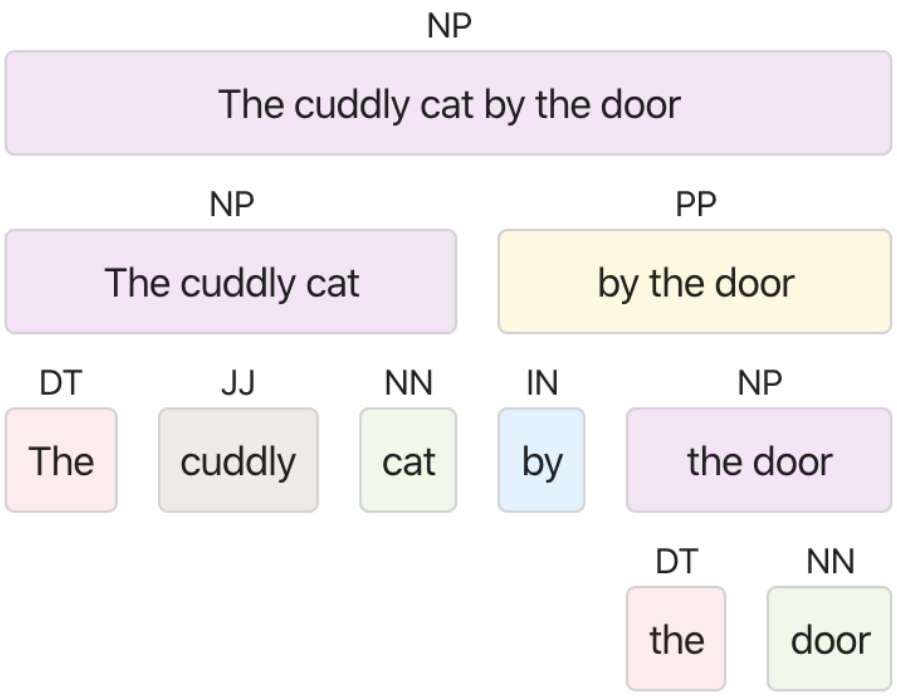
the cuddly cat, by the door

NP NP PP

Phrases can combine into bigger phrases:

the cuddly cat by the door

NP NP PP NP



Credits to: C. Manning

Constituents

- In general, **words forming <u>constituents</u>** can move around together, e.g.:
  - ✓ **<u>On 7th September</u>** I'd like to fly from London to New York.
  - ✓ I'd like to fly from London to New York **<u>on 7th September</u>**.
  - ✗ **<u>On September</u>** I'd like to fly from London to New York **<u>7th</u>**.

- E.g.: a **Prepositional Phrase (PP)** would need a **preposition** followed by a **noun** (e.g. "In September", "on time")

Constituents

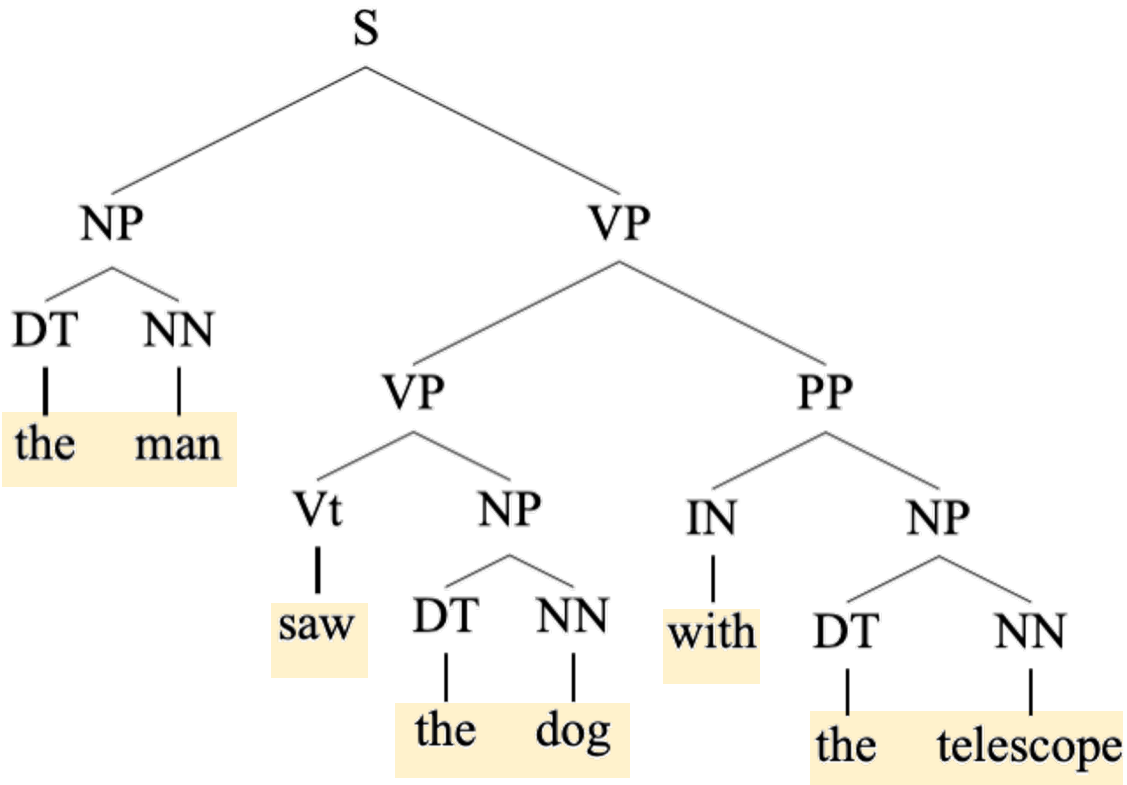
Examples of constituents:

- Noun Phrase (**NP**):
  the flights, some flights, any flights,...
- Prepositional Phrase (**PP**):
  in the morning, at home,...
- Verb Phrase (**VP**):

VP → Verb	e.g. disappear
VP → Verb NP	e.g. prefer a morning flight
VP → Verb NP PP	e.g. leave London in the morning
VP → Verb PP	e.g. leaving on Thursday

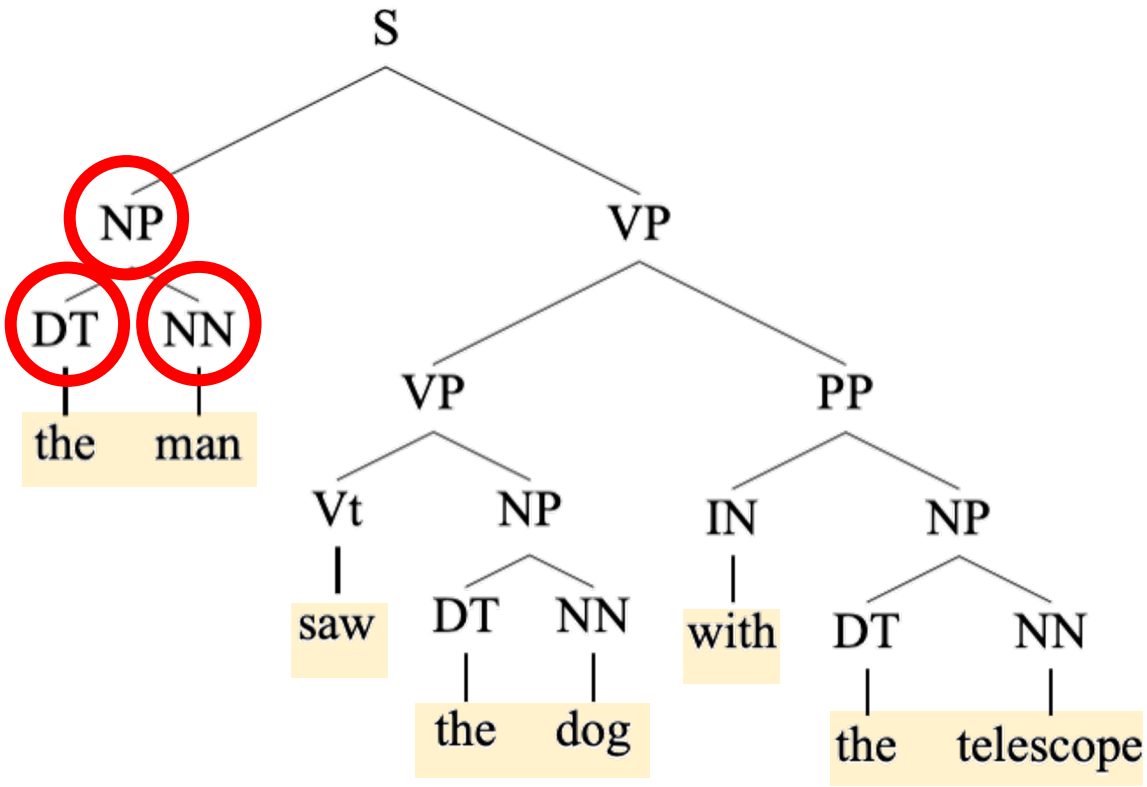
Constituency (Phrase Structure)

- " The man saw the dog with the telescope. "
- What do we understand here?



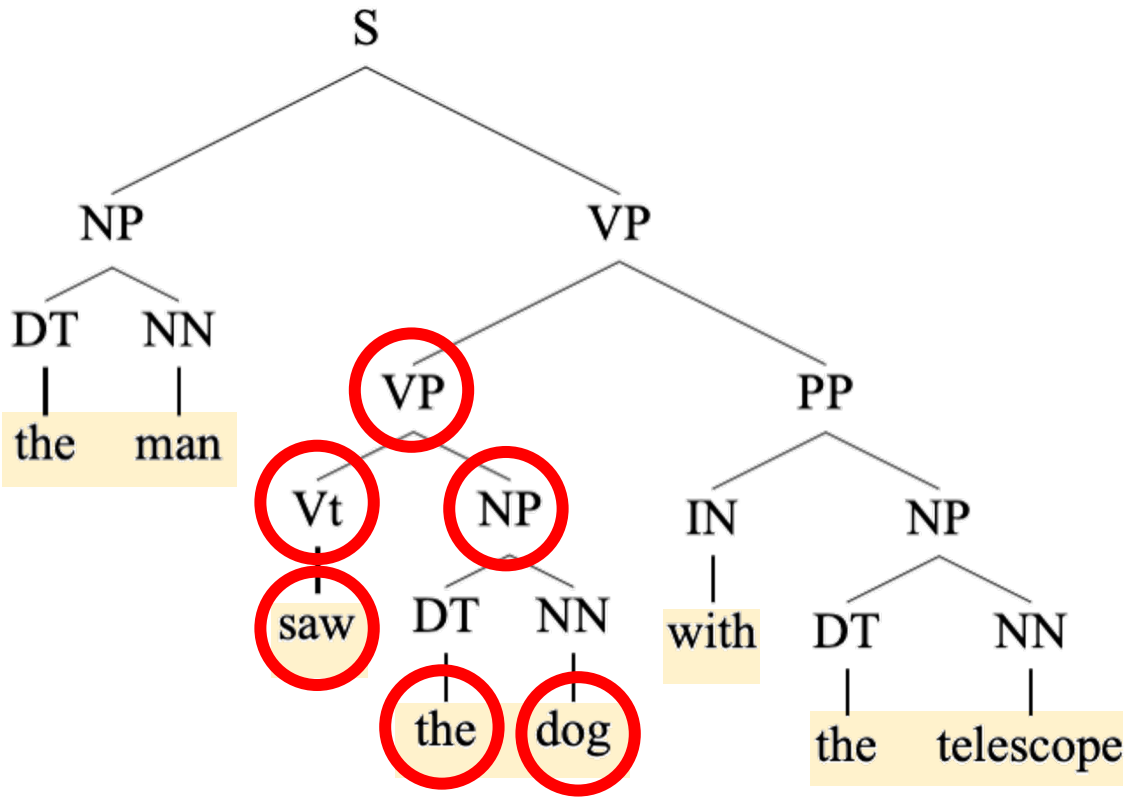
Constituency (Phrase Structure)

- " The man saw the dog with the telescope. "
- What do we understand here?



Constituency (Phrase Structure)

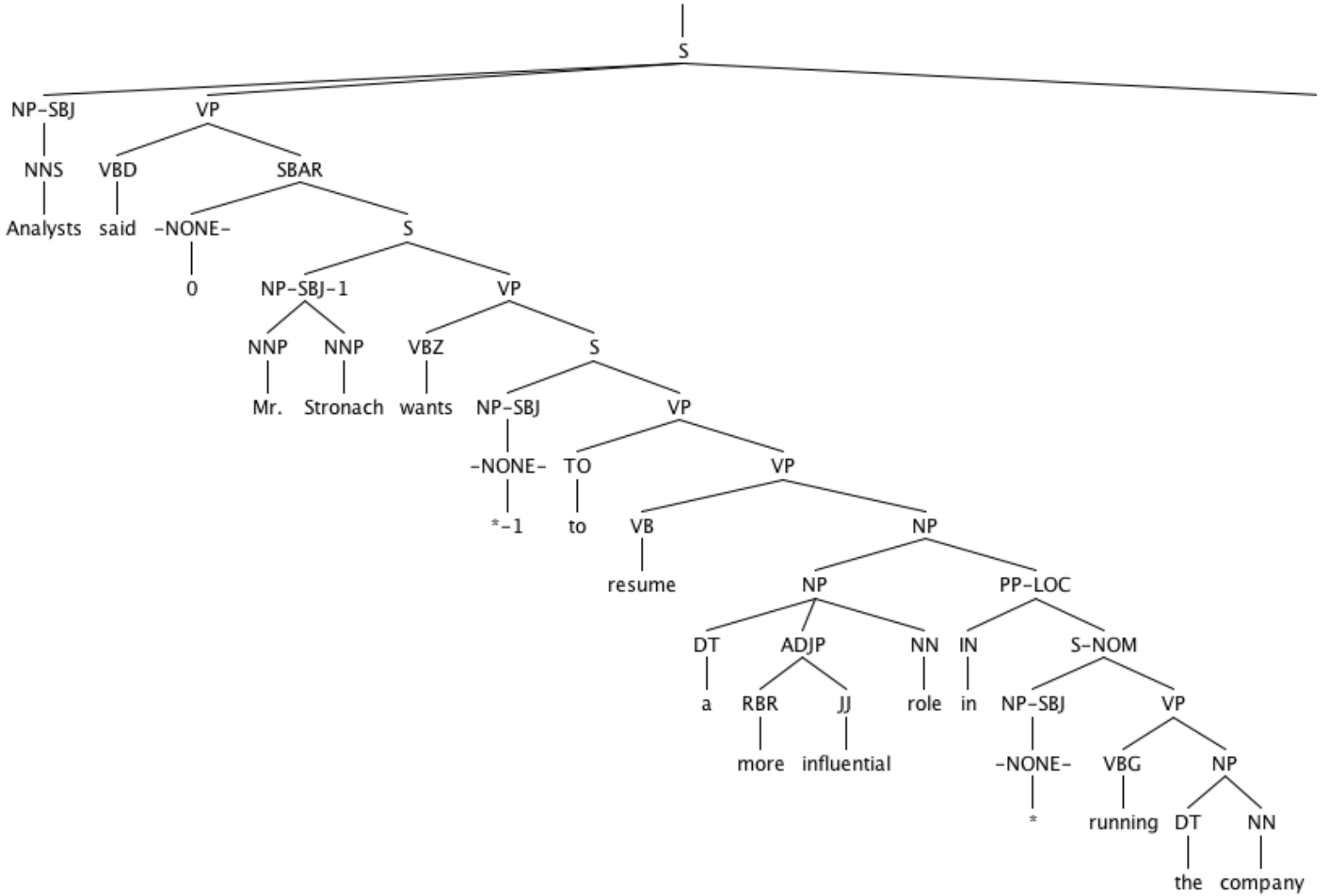
- " The man saw the dog with the telescope. "
- What do we understand here?



VP → Verb NP

Constituency (Phrase Structure)

" Analysts said Mr. Stronach wants to resume a more influential role in running the company. "



Two types of syntactical structures

1. **Constituency** structures
2. **Dependency** structures

Two types of syntactical structures

1. **Constituency** structures

2. **Dependency** structures

2. Dependency structures

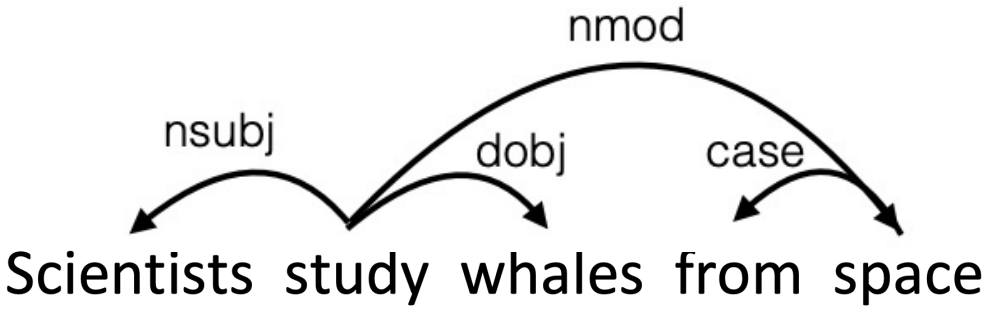
- **Dependency structure** shows which words depend on (modify or are arguments of) which other words.

Scientists study whales from space

✓

2. Dependency structures

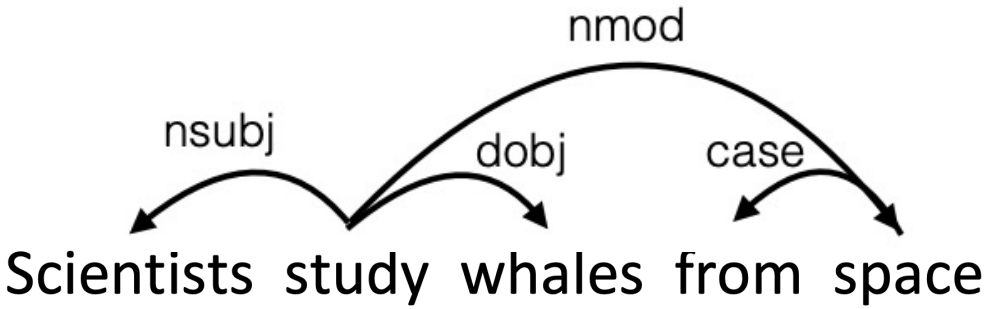
- **Dependency structure** shows which words depend on (modify or are arguments of) which other words.

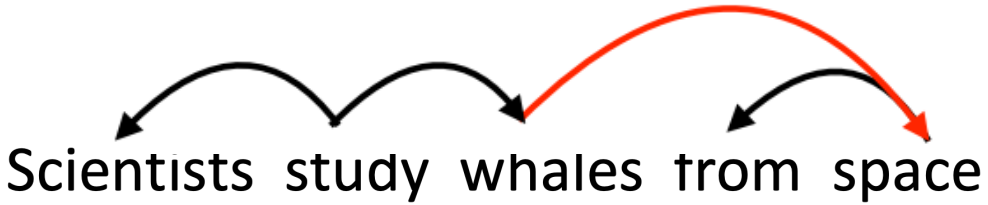


✓

2. Dependency structures

- **Dependency structure** shows which words depend on (modify or are arguments of) which other words.

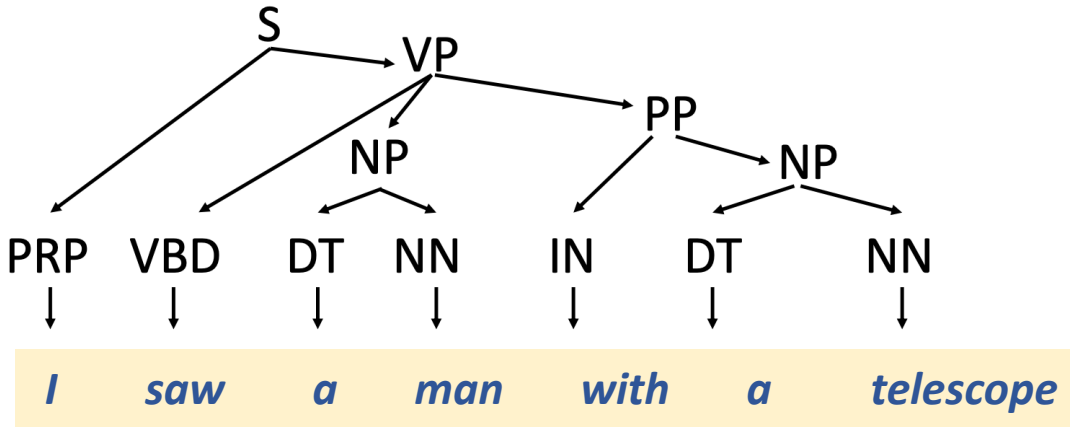




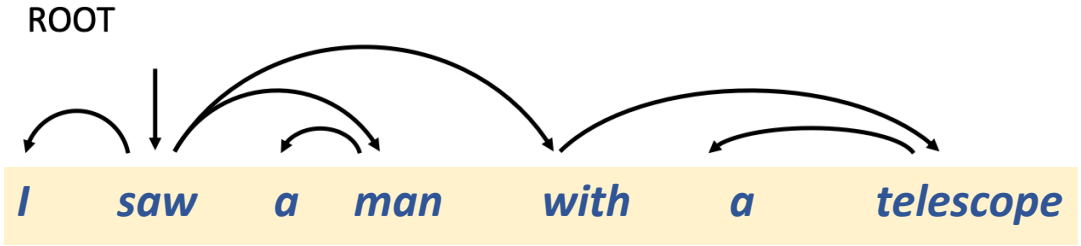
Credits to: C. Manning

Two types of syntactical structures

1. **Constituency** structures



2. **Dependency** structures



Two types of syntactical structures

1. **Constituency** structures

How?

2. **Dependency** structures

How?

Two types of syntactical structures

1. **Constituency** structures
   - Phrase Structure Grammar
   - Context-Free Grammars (CFGs) → and Lexicalised Grammars

Cocke-Kasami-Younger (CKY) algorithm

2. **Dependency** structures

How?

Two types of syntactical structures

1. **Constituency** structures
   - Phrase Structure Grammar
   - Context-Free Grammars (CFGs) → and Lexicalised Grammars

Cocke-Kasami-Younger (CKY) algorithm

2. **Dependency** structures
   - Graph Algorithms
   - Transition-based Algorithms

Two types of syntactical structures

1. **Constituency** structures
   - Phrase Structure Grammar
   - Context-Free Grammars (CFGs) → and Lexicalised Grammars

Cocke-Kasami-Younger (CKY) algorithm

2. **Dependency** structures
   - Graph Algorithms
   - Transition-based Algorithms

Context-Free Grammars (CFG)

- **Context-free grammars (CFGs)** are widely used **formal system** for modelling constituency structure in natural languages.

- They consist of:
  - Terminals
  - Non-terminals
  - Rules

- which allow us to produce grammatical sentences

CFG Example

Rules

N → children
N → candy
V → buy
N → money
P → with
V → eat

Terminals

CFG Example

Rules

S → NP VP
VP → V NP PP
NP → N
PP → P NP

N → children
N → candy
V → buy
N → money
P → with
V → eat

Non-terminals

Terminals

CFG Example

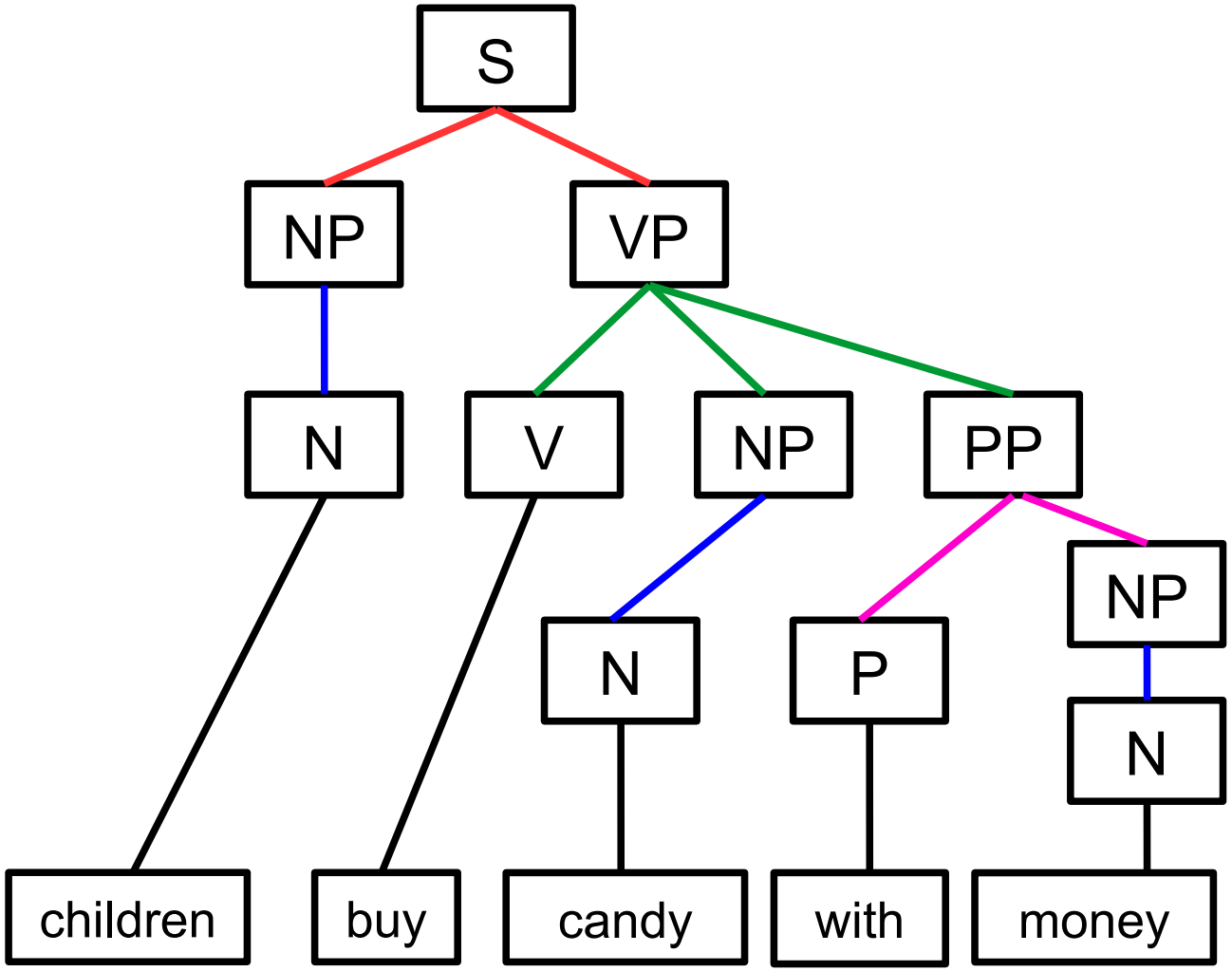
S → NP VP
VP → V NP PP
NP → N
PP → P NP

N → children
N → candy
V → buy
N → money
P → with
V → eat

CFG Example

S → NP VP
VP → V NP PP
NP → N
PP → P NP

N → children
N → candy
V → buy
N → money
P → with
V → eat



Probabilistic CFG (PCFG)

- Like CFG, but each rule has a **probability**.

S → NP VP	N → children
VP → V NP	N → candy
VP → V NP PP	N → money
…	…

Probabilistic CFG (PCFG)

- Like CFG, but each rule has a **probability**.

S → NP VP (1.0)	N → children
VP → V NP	N → candy
VP → V NP PP	N → money
…	…

Probabilistic CFG (PCFG)

- Like CFG, but each rule has a **probability**.

S → NP VP	(1.0)	
VP → V NP	(0.6)	1.0
VP → V NP PP	(0.4)	
…		

N → children
N → candy
N → money
…

Probabilistic CFG (PCFG)

- Like CFG, but each rule has a **probability**.

S → NP VP	(1.0)	N → children	(0.5)
VP → V NP	(0.6)	N → candy	(0.3)
VP → V NP PP	(0.4)	N → money	(0.2)
…		…	

1.0

Probabilistic CFG (PCFG)

- Like CFG, but each rule has a **probability**.

S → NP VP	(1.0)		N → children	(0.5)	
VP → V NP	(0.6)	1.0	N → candy	(0.3)	1.0
VP → V NP PP	(0.4)		N → money	(0.2)	
…			…		

Parsing

- We may need to work in both directions:

  - Top-down approach: from rules to generate sentences
    → **Natural Language Generation**

  - Bottom-up approach: from sentences to generating trees
    → **Parsing**

Parsing

- We are given a sentence, and we need to **generate the tree** based on the rules we have.

Cocke-Kasami-Younger (CKY) algorithm for parsing

- CKY requirement: grammars used with it must be in **Chomsky Normal Form** (CNF)
  - Rules of the form $\boxed{A \rightarrow B\ C}$ or $\boxed{A \rightarrow w}$
  - i.e., the right-hand side (RHS) of each rule must expand either to two non-terminals or to a single terminal

- Restricting a grammar to CNF does not lead to any loss in expressiveness
  - Any CFG can be **converted** into a corresponding CNF grammar

CKY Parsing Algorithm

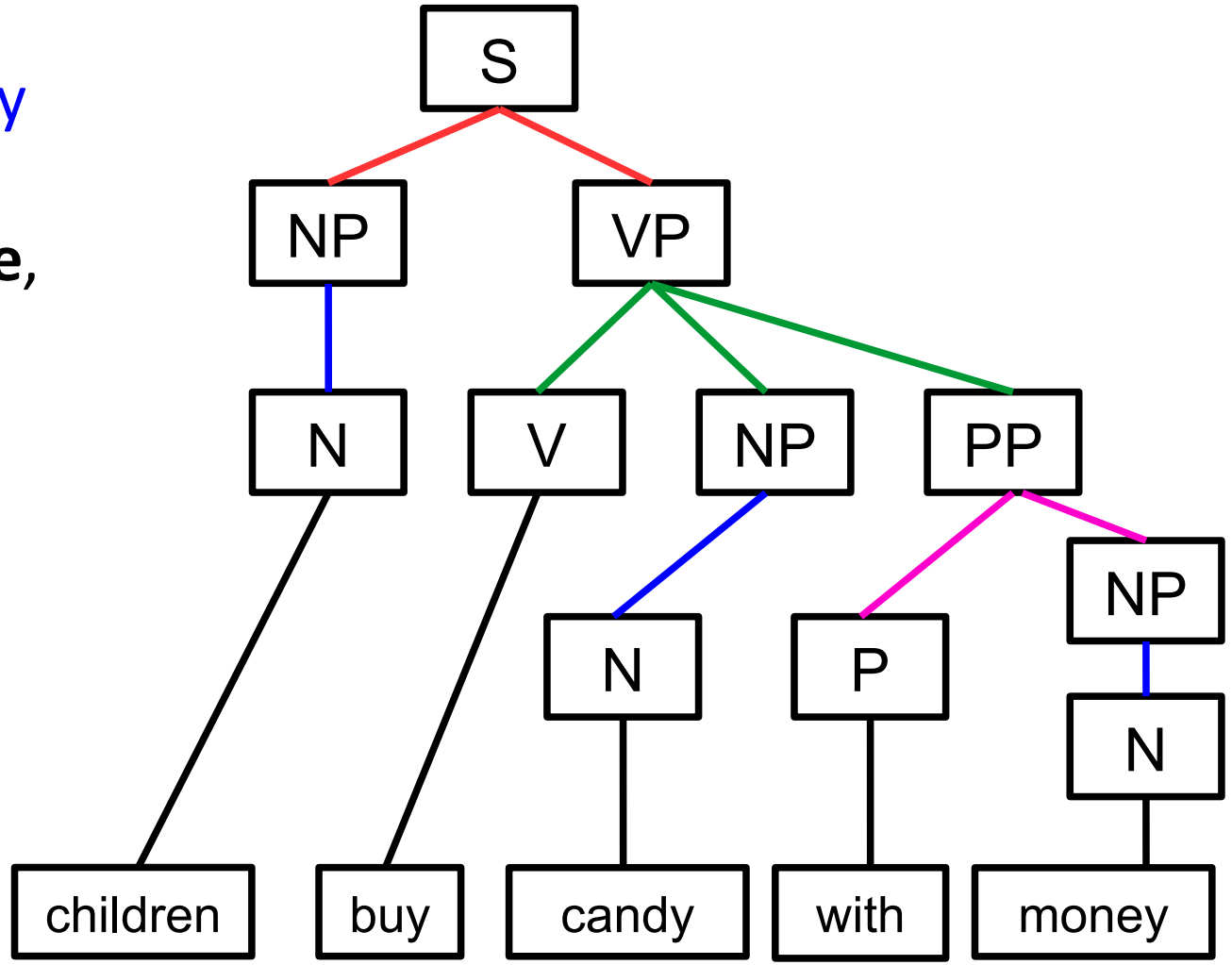
- We have the sentence:
  children buy candy with money

  We aim to **infer the whole tree**,
  as in the picture on the right.

CKY Parsing Algorithm

- We have the sentence:
  children buy candy with money

  We aim to **infer the whole tree**, as in the picture on the right.



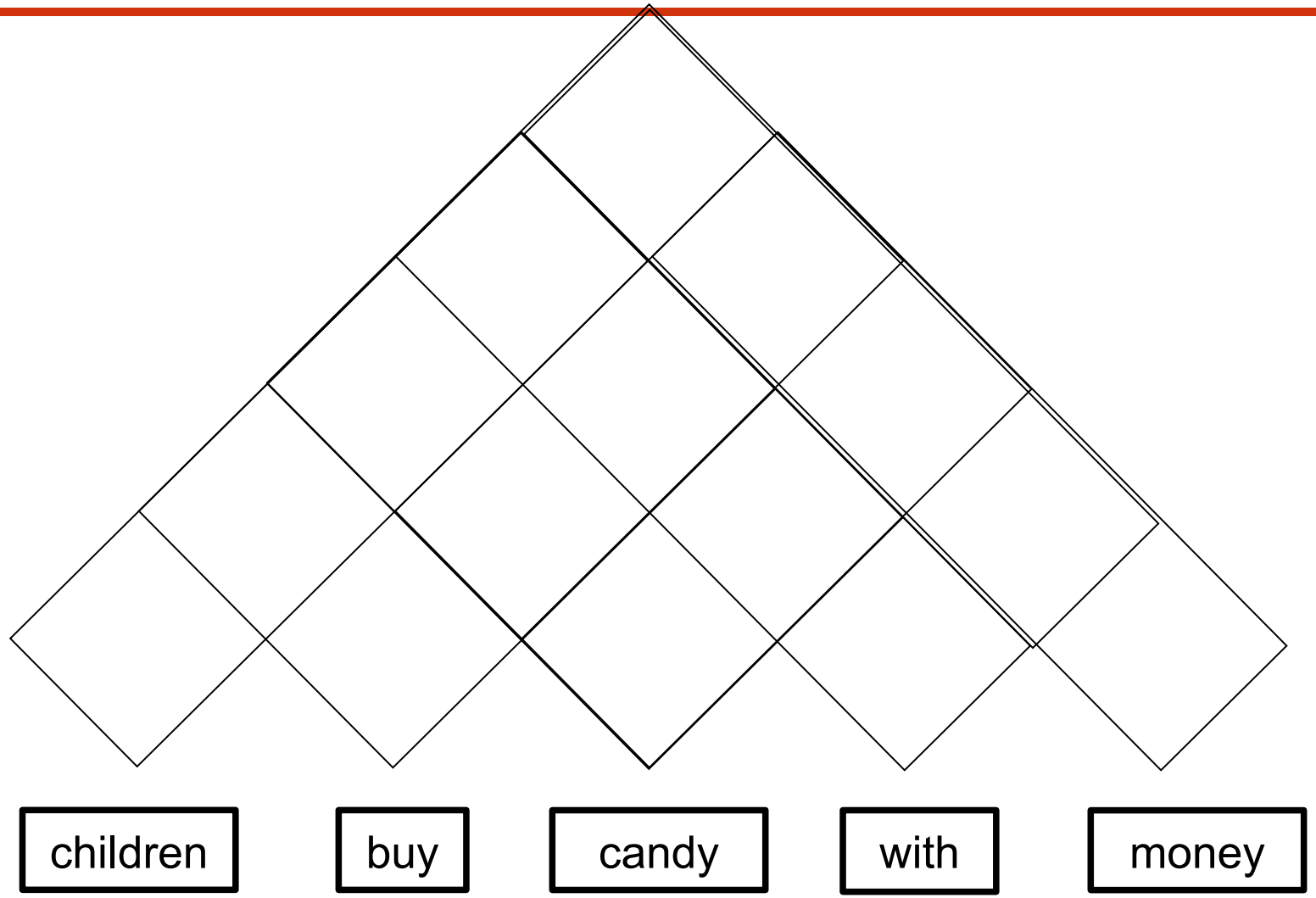
CKY Parsing Algorithm

- Having the entire constituency list (T, N, R)
  - Use **bottom-up approach** to fill in the tree

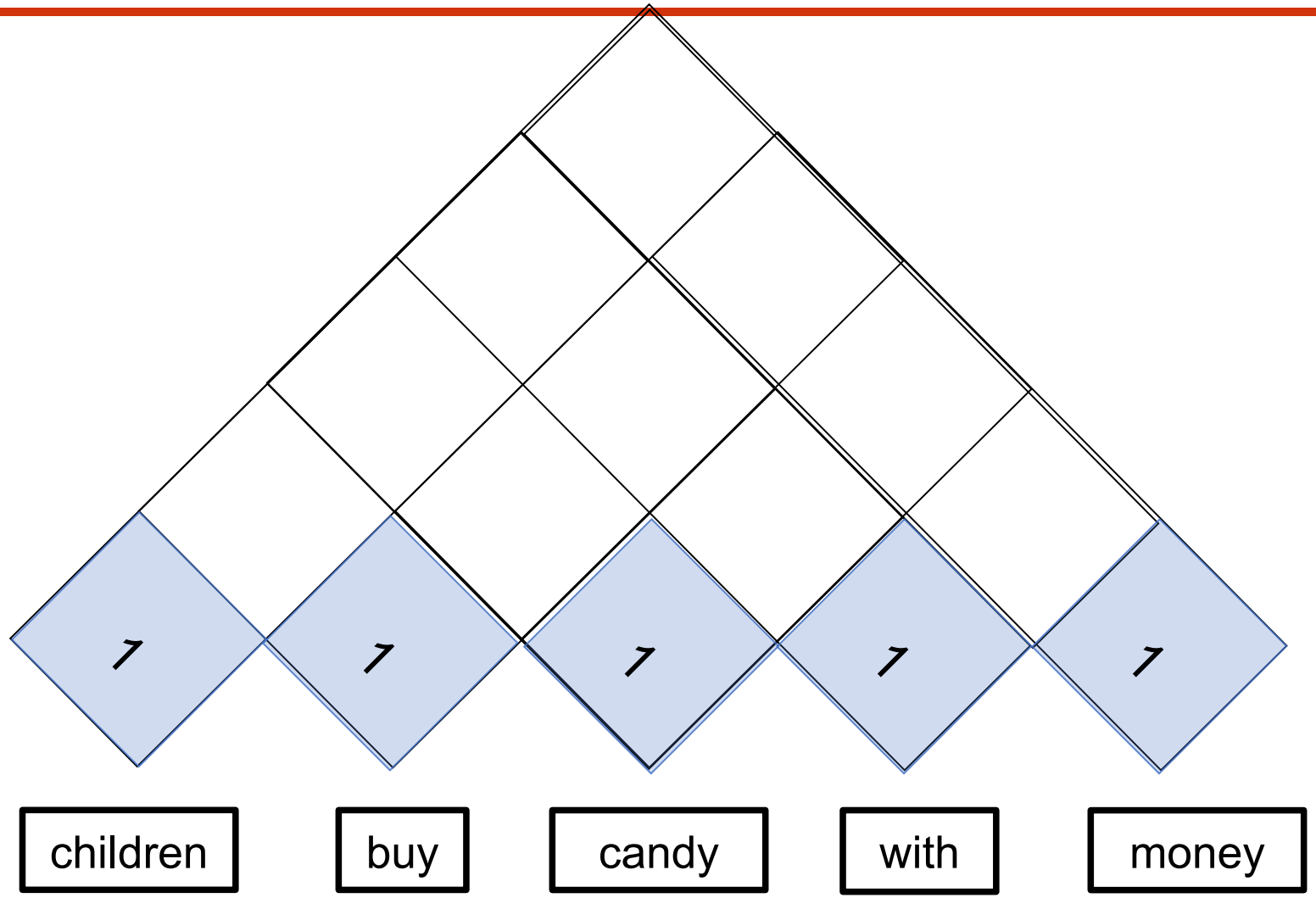
CKY Parsing Algorithm

children	buy	candy	with	money

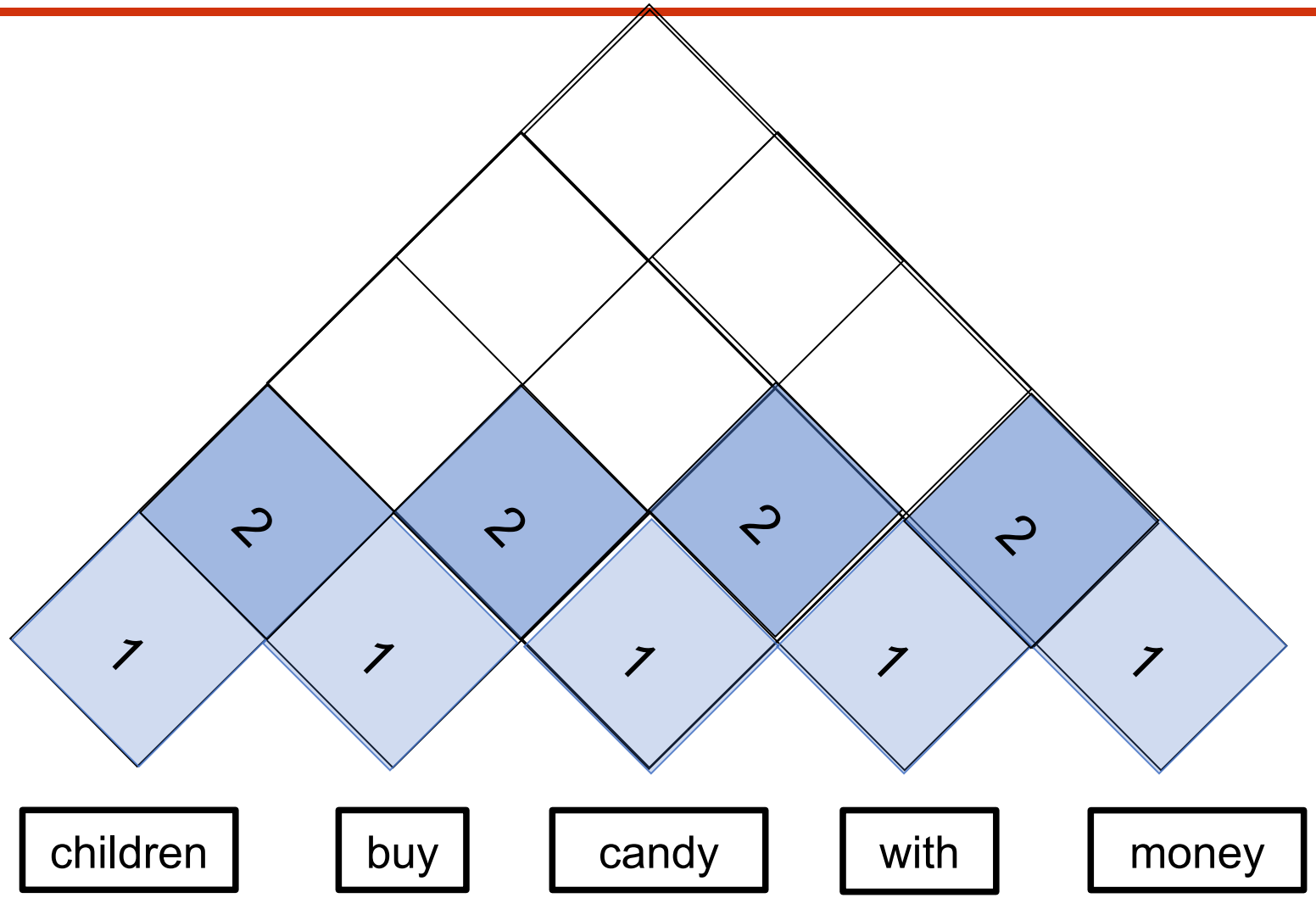
CKY Parsing Algorithm



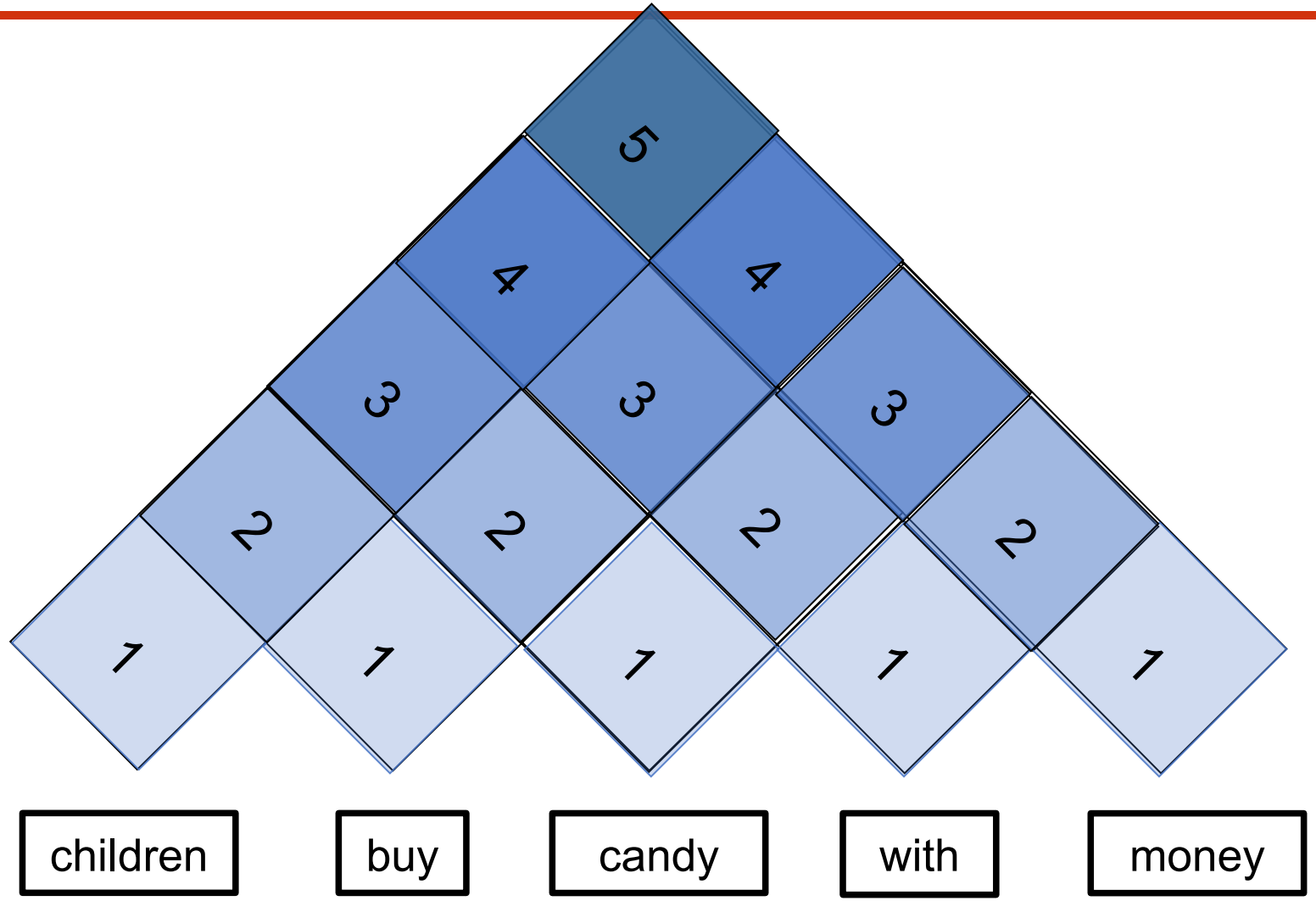
CKY Parsing Algorithm



CKY Parsing Algorithm

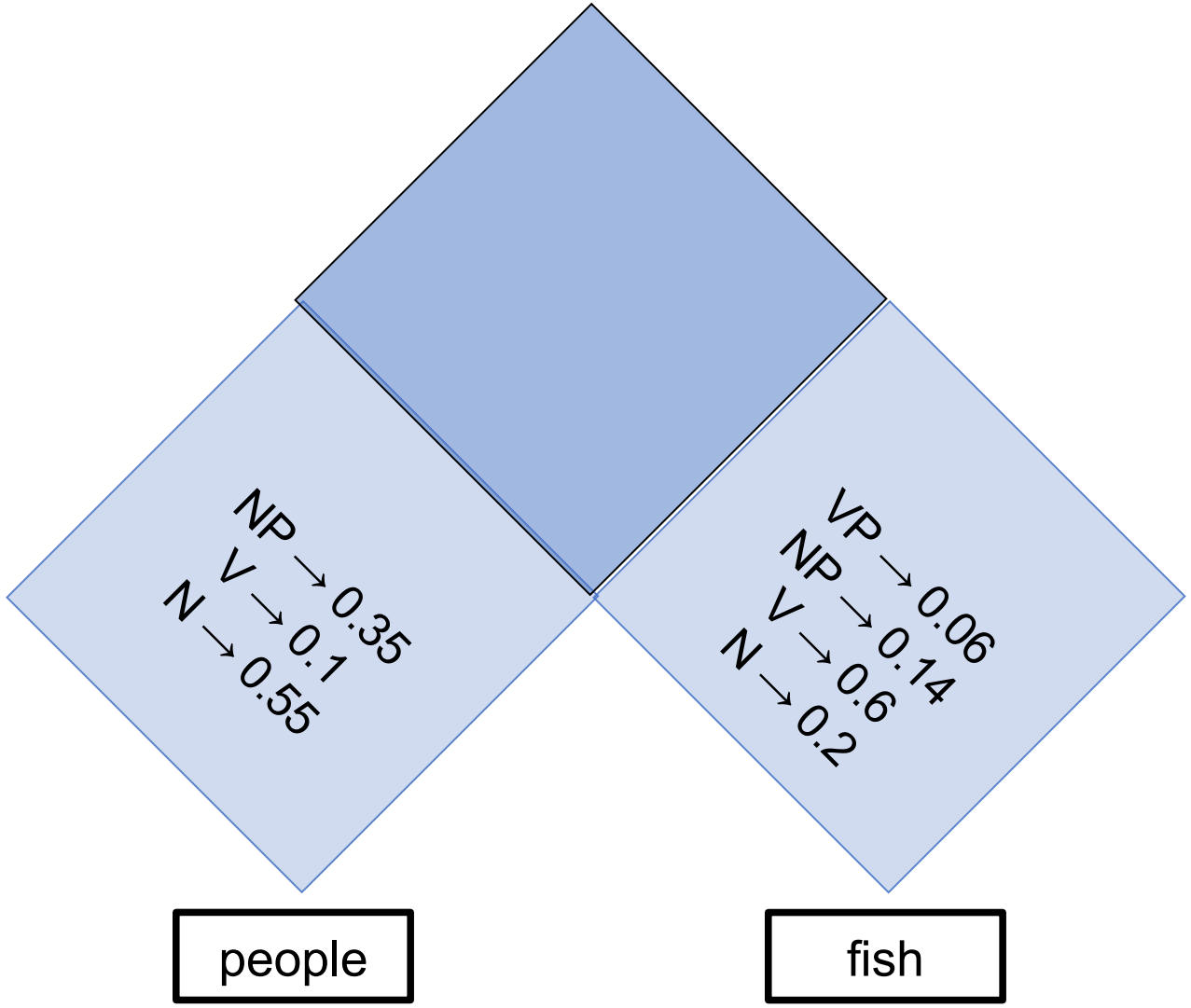


CKY Parsing Algorithm



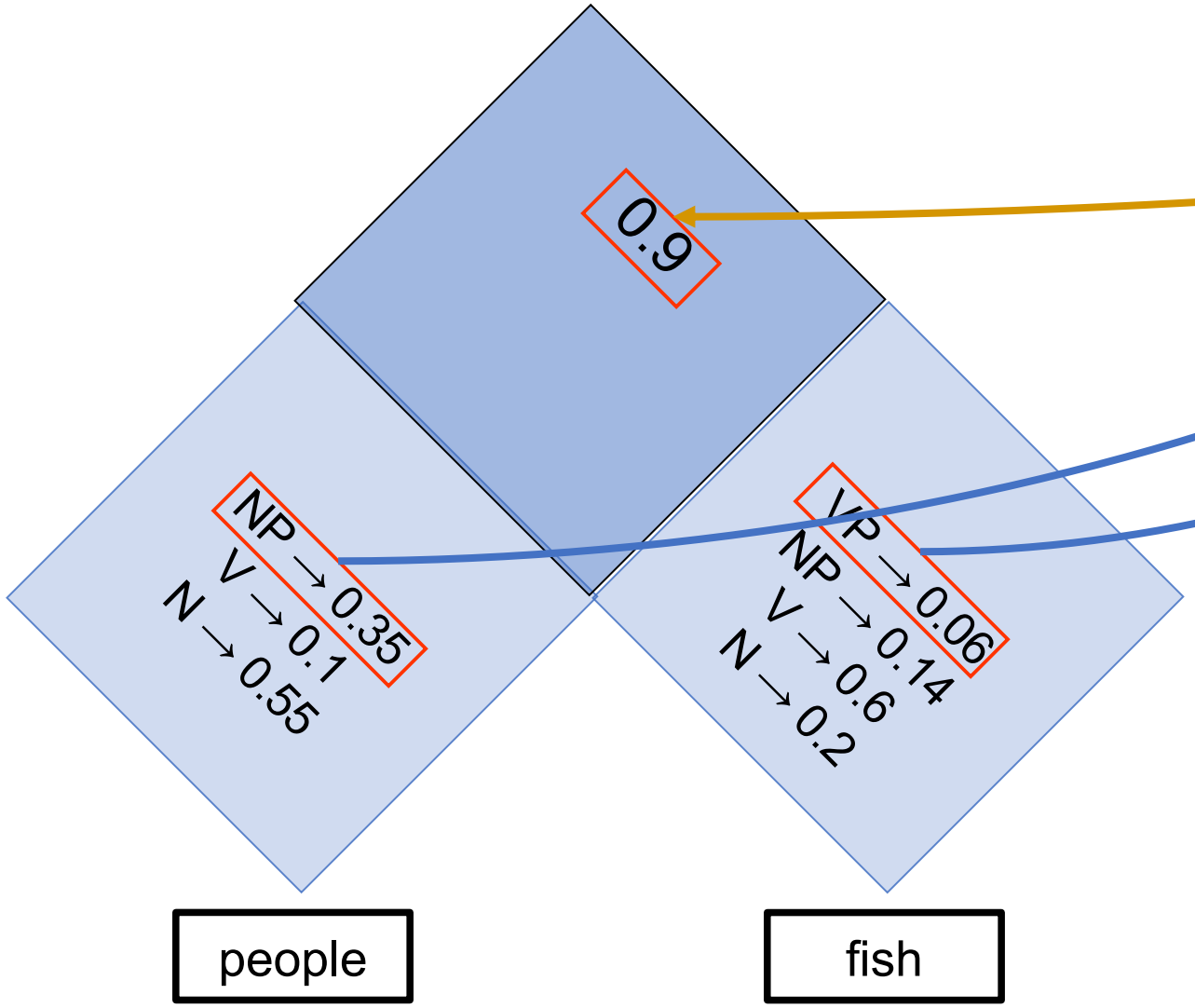
CKY Parsing Algorithm

VP → V NP PP



S → NP VP	0.9
S → VP	0.1
VP → V NP	0.5
VP → V	0.1
VP → V @VP_V	0.3
VP → V PP	0.1
@VP_V → NP PP	1.0
NP → NP NP	0.1
NP → NP PP	0.2
NP → N	0.7
PP → P NP	1.0

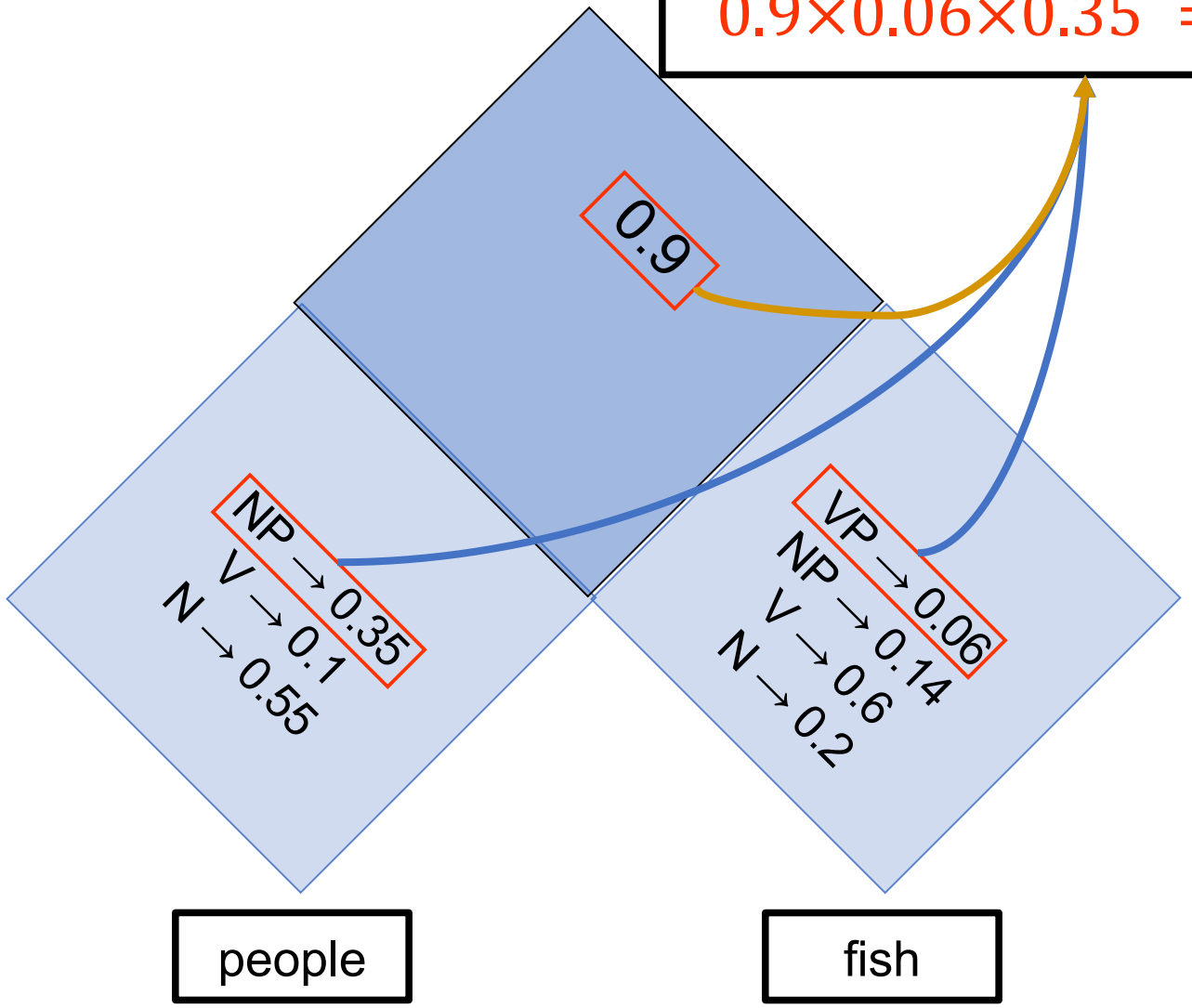
CKY Parsing Algorithm



S → NP VP	0.9
S → VP	0.1
VP → V NP	0.5
VP → V	0.1
VP → V @VP_V	0.3
VP → V PP	0.1
@VP_V → NP PP	1.0
NP → NP NP	0.1
NP → NP PP	0.2
NP → N	0.7
PP → P NP	1.0

CKY Parsing Algorithm

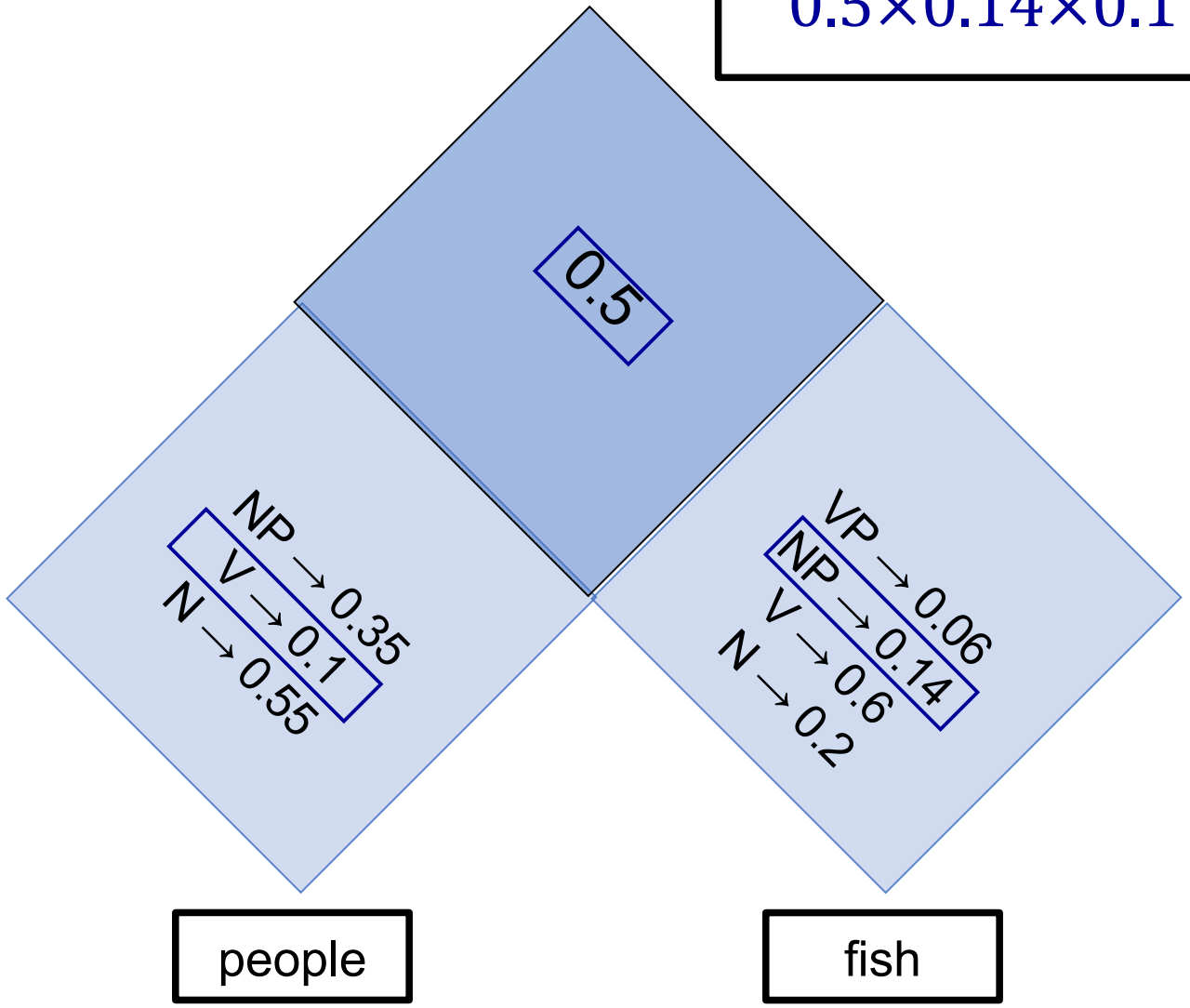
$$0.9 \times 0.06 \times 0.35 = 0.0189$$



S → NP VP	0.9
S → VP	0.1
VP → V NP	0.5
VP → V	0.1
VP → V @VP_V	0.3
VP → V PP	0.1
@VP_V → NP PP	1.0
NP → NP NP	0.1
NP → NP PP	0.2
NP → N	0.7
PP → P NP	1.0

CKY Parsing Algorithm

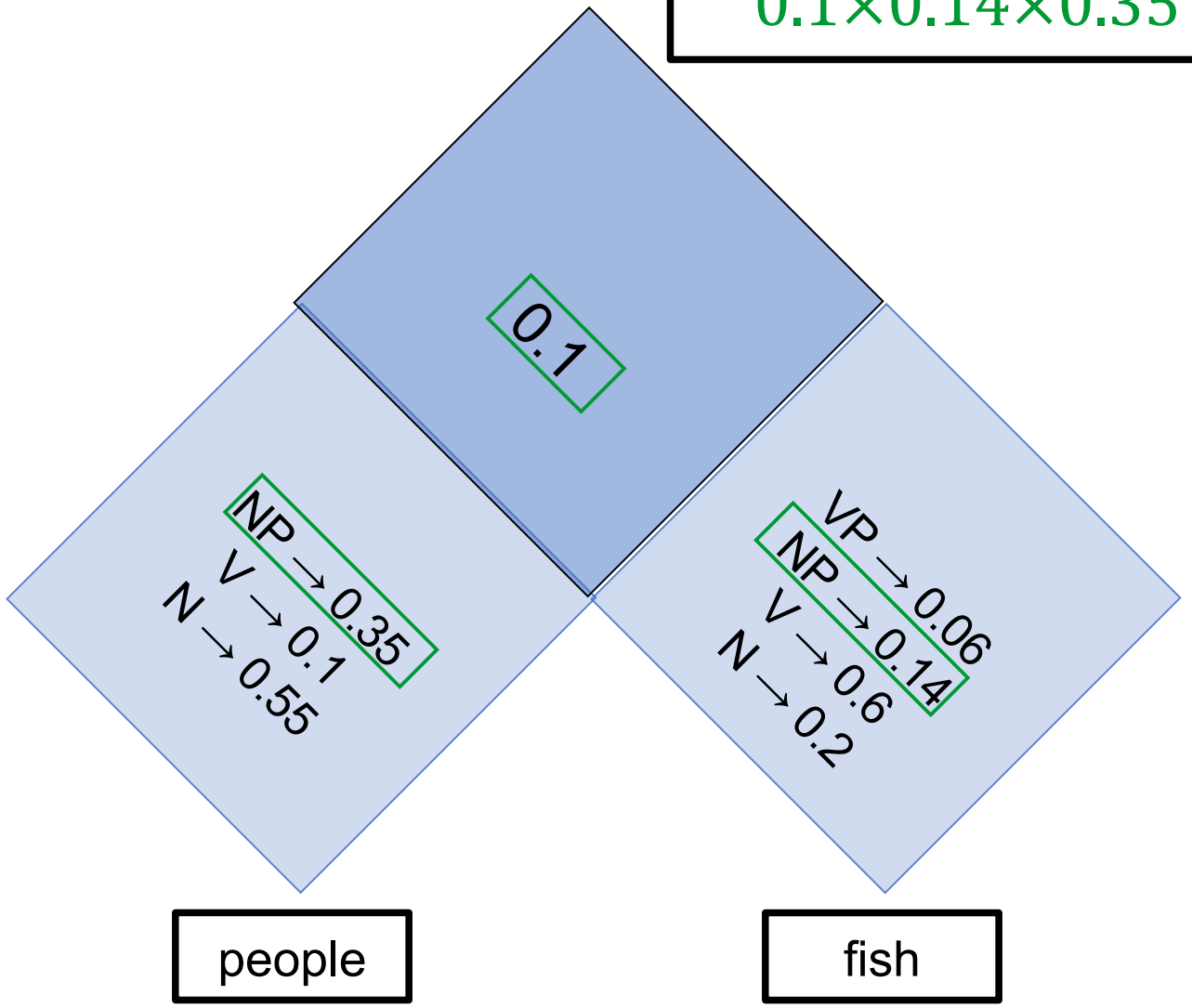
$$0.5 \times 0.14 \times 0.1 \;=\; 0.007$$



S → NP VP	0.9
S → VP	0.1
VP → V NP	0.5
VP → V	0.1
VP → V @VP_V	0.3
VP → V PP	0.1
@VP_V → NP PP	1.0
NP → NP NP	0.1
NP → NP PP	0.2
NP → N	0.7
PP → P NP	1.0

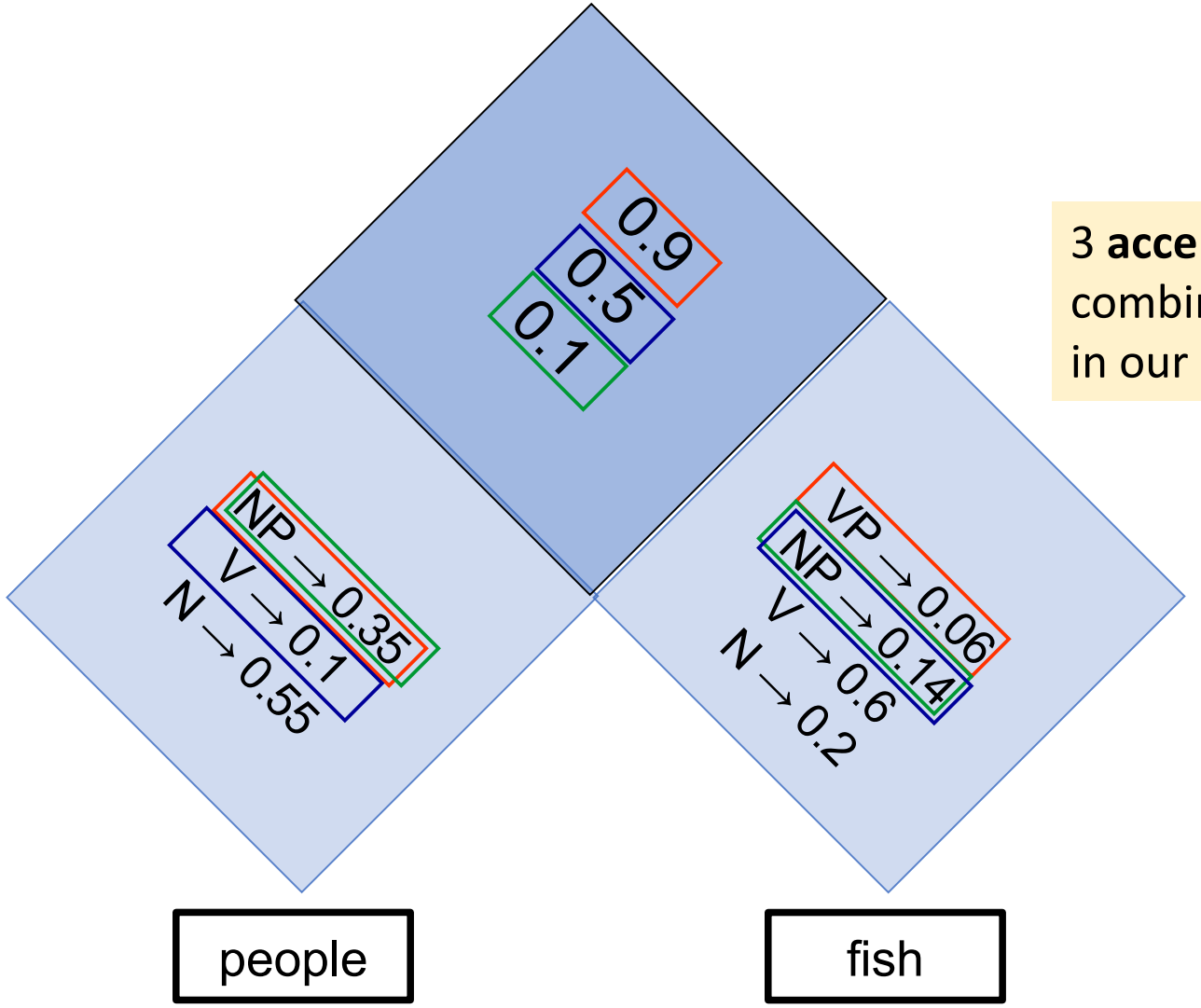
CKY Parsing Algorithm

$$0.1 \times 0.14 \times 0.35 = 0.049$$



S → NP VP	0.9
S → VP	0.1
VP → V NP	0.5
VP → V	0.1
VP → V @VP_V	0.3
VP → V PP	0.1
@VP_V → NP PP	1.0
NP → NP NP	0.1
NP → NP PP	0.2
NP → N	0.7
PP → P NP	1.0

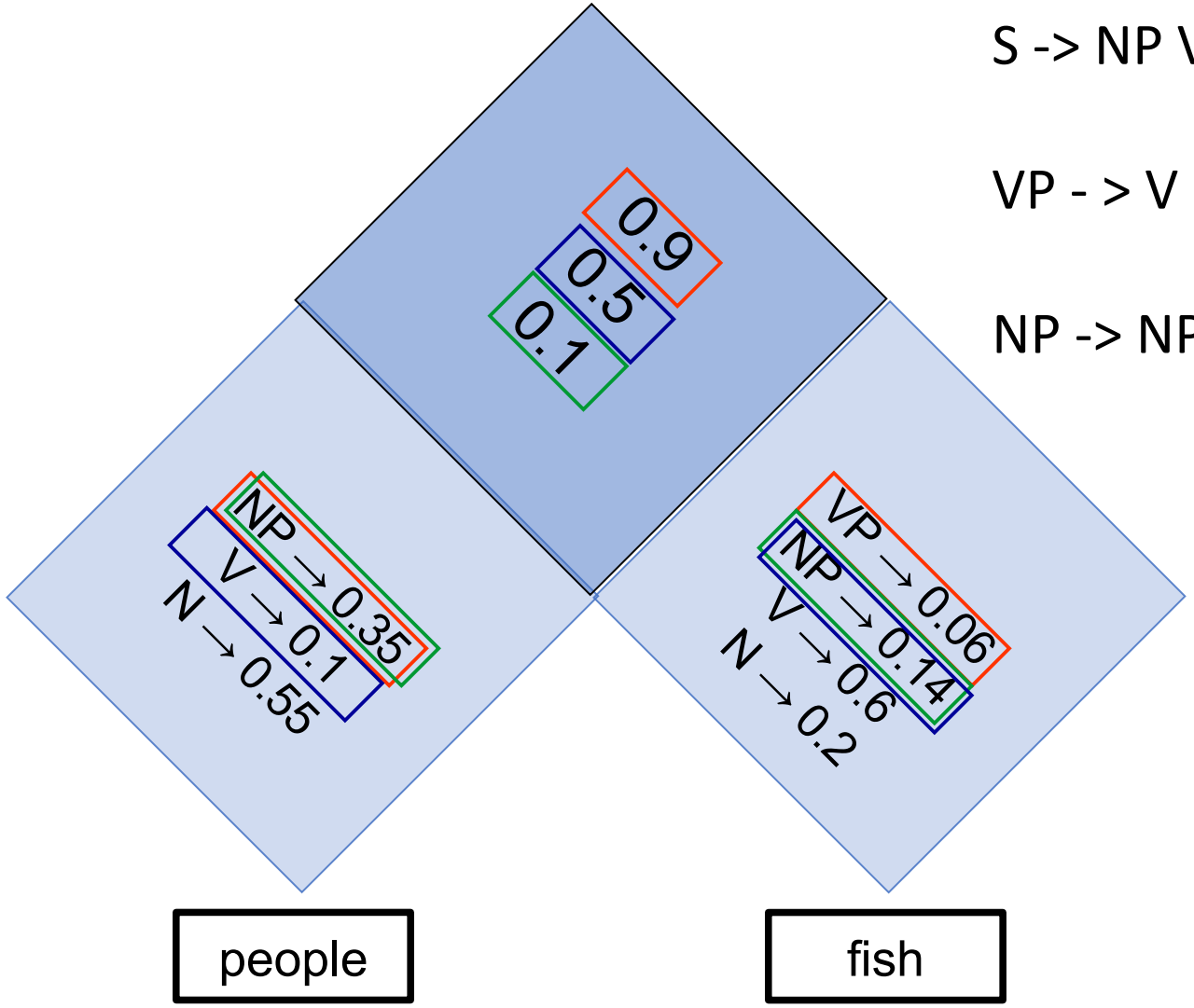
CKY Parsing Algorithm



3 **acceptable** combinations in our grammar

S → NP VP	0.9
S → VP	0.1
VP → V NP	0.5
VP → V	0.1
VP → V @VP_V	0.3
VP → V PP	0.1
@VP_V → NP PP	1.0
NP → NP NP	0.1
NP → NP PP	0.2
NP → N	0.7
PP → P NP	1.0

CKY Parsing Algorithm



S -> NP VP

VP - > V NP

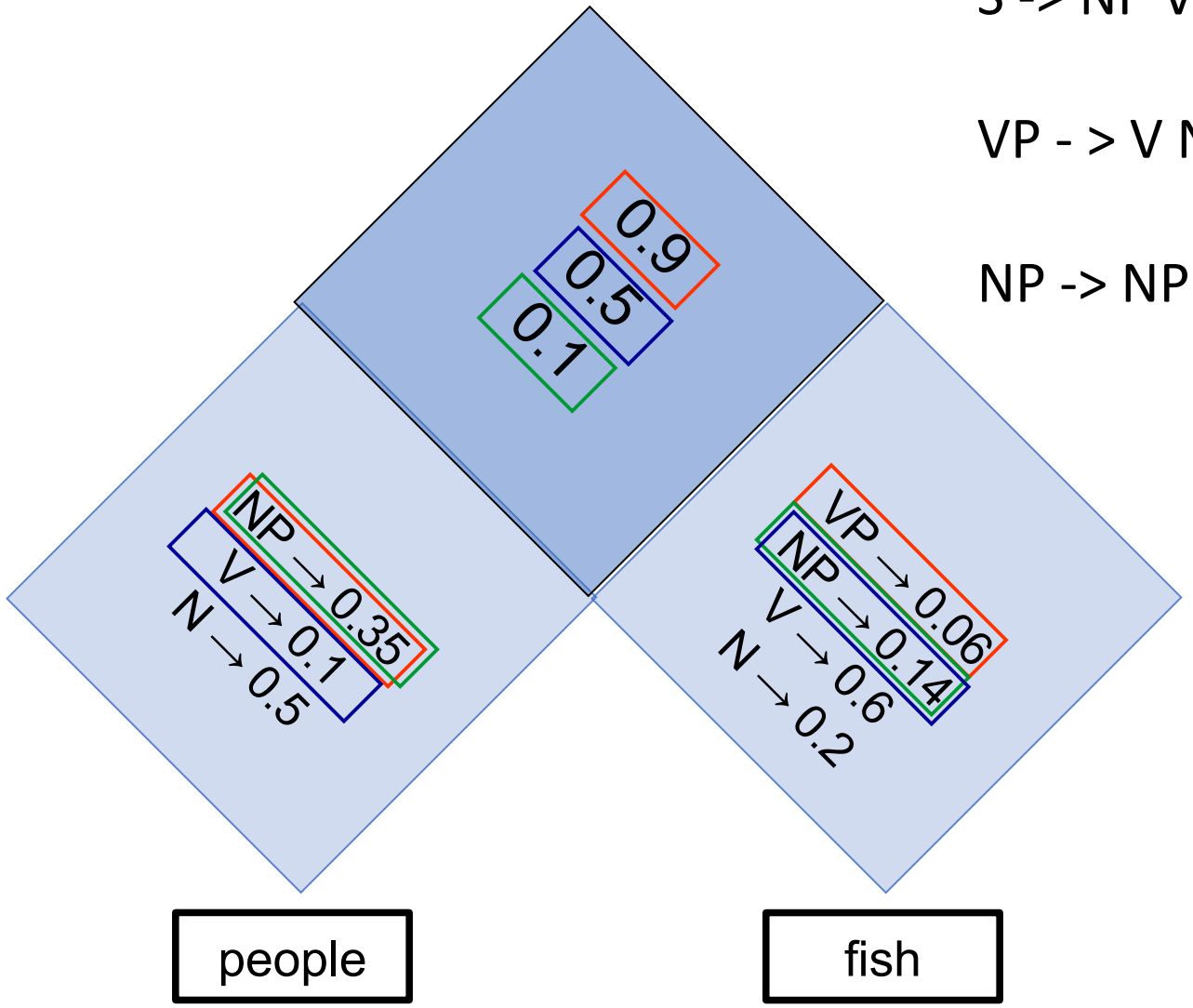
NP -> NP NP

$$0.9 \times 0.06 \times 0.35 = 0.0189$$

$$0.5 \times 0.14 \times 0.1 = 0.007$$

$$0.1 \times 0.14 \times 0.35 = 0.049$$

CKY Parsing Algorithm



S -> NP VP

VP - > V NP

NP -> NP NP

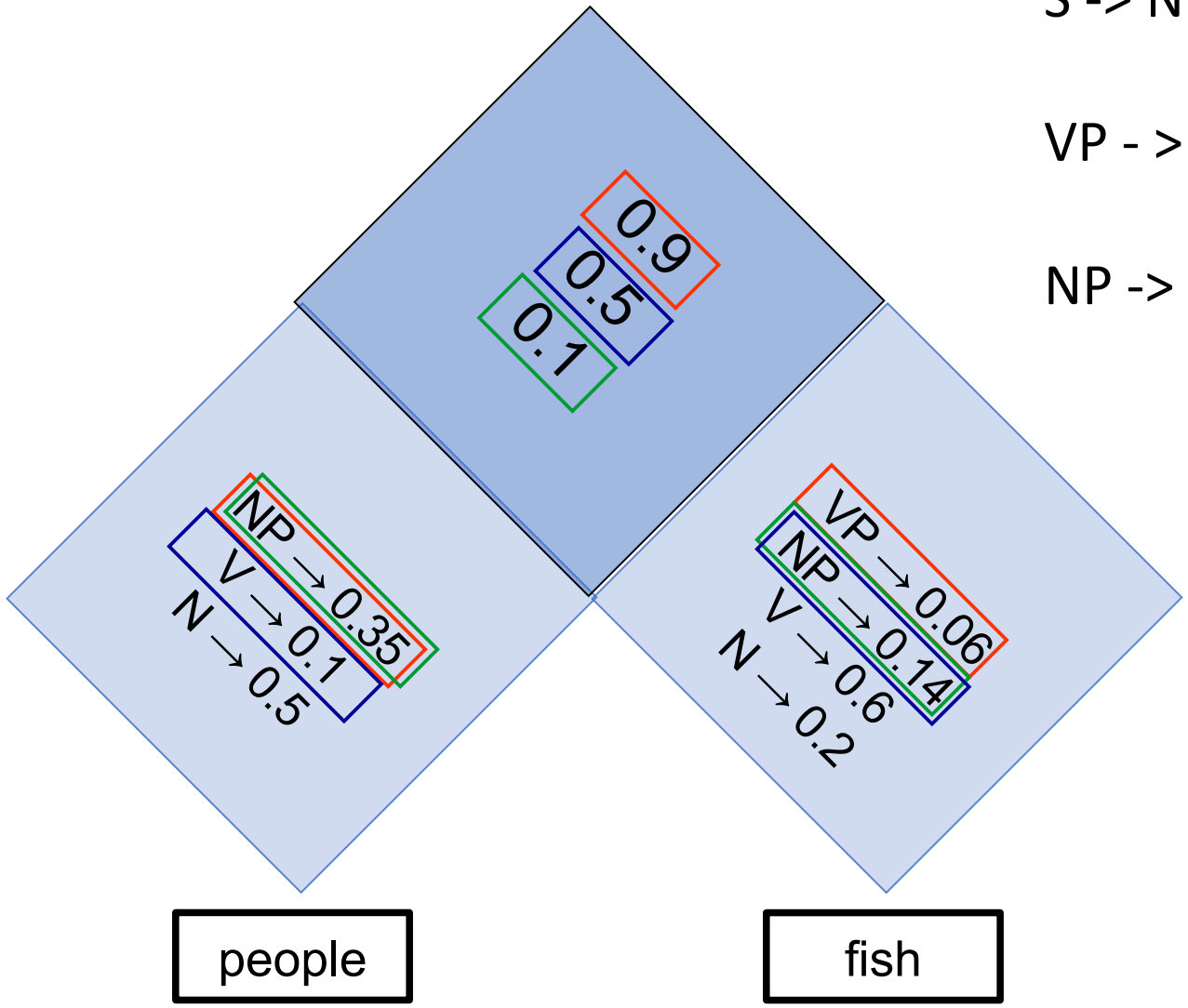
$0.9 \times 0.06 \times 0.35 = 0.0189$

$0.5 \times 0.14 \times 0.1 = 0.007$

$0.1 \times 0.14 \times 0.35 = 0.049$

The Viterbi (maximum) score determines the predicted parsed tree via CKY.

CKY Parsing Algorithm



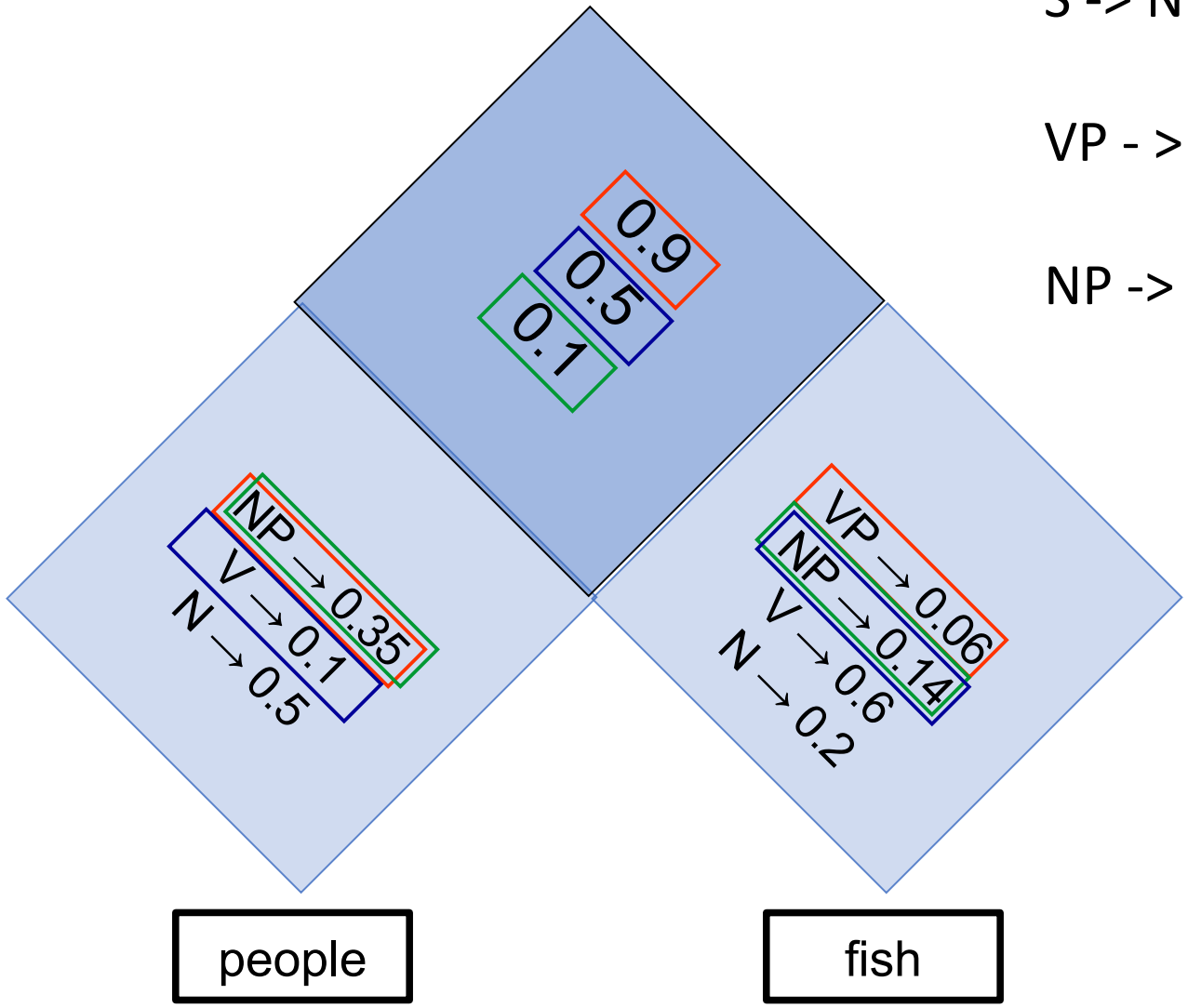
S -> NP VP $0.9 \times 0.06 \times 0.35 = 0.0189$

VP - > V NP $0.5 \times 0.14 \times 0.1 = 0.007$

NP -> NP NP $0.1 \times 0.14 \times 0.35 = 0.049$

It's larger, but doesn't belong to a "S → ..." rule (i.e. whole sentence)

CKY Parsing Algorithm



S -> NP VP $0.9 \times 0.06 \times 0.35 = 0.0189$

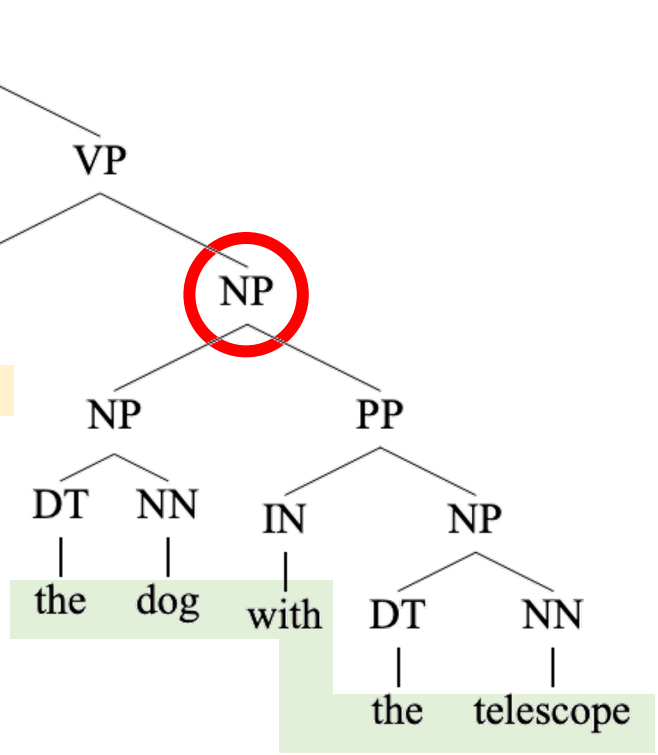
VP - > V NP $0.5 \times 0.14 \times 0.1 = 0.007$

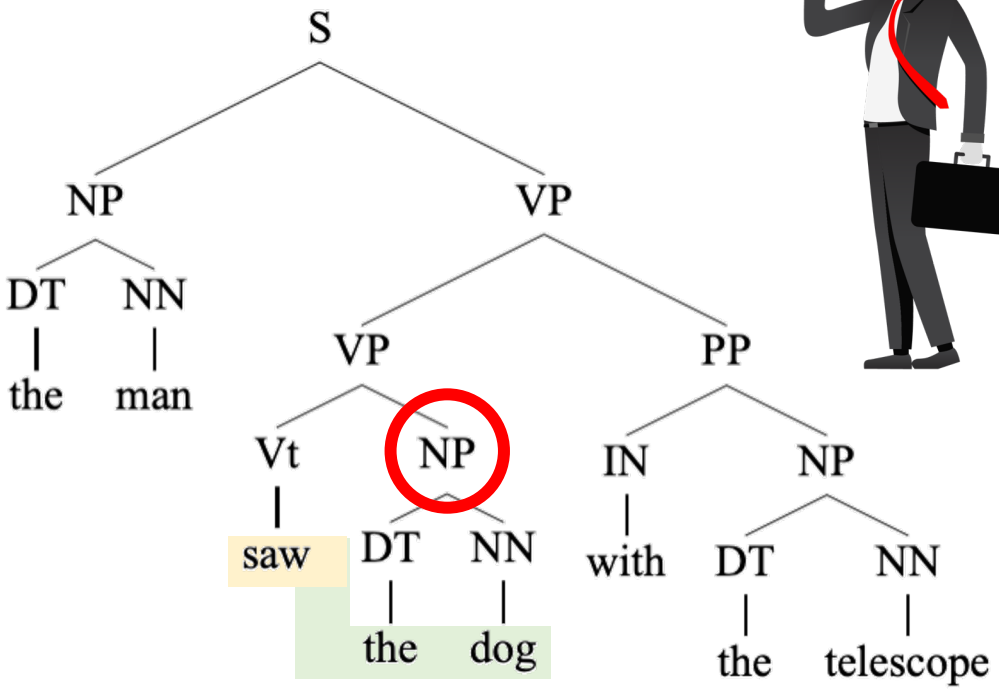
NP -> NP NP $0.1 \times 0.14 \times 0.35 = 0.049$

If there were more levels (upwards) in the tree, then we'd choose this one, and carry on.

Key Challenge in Parsing: **Ambiguity**

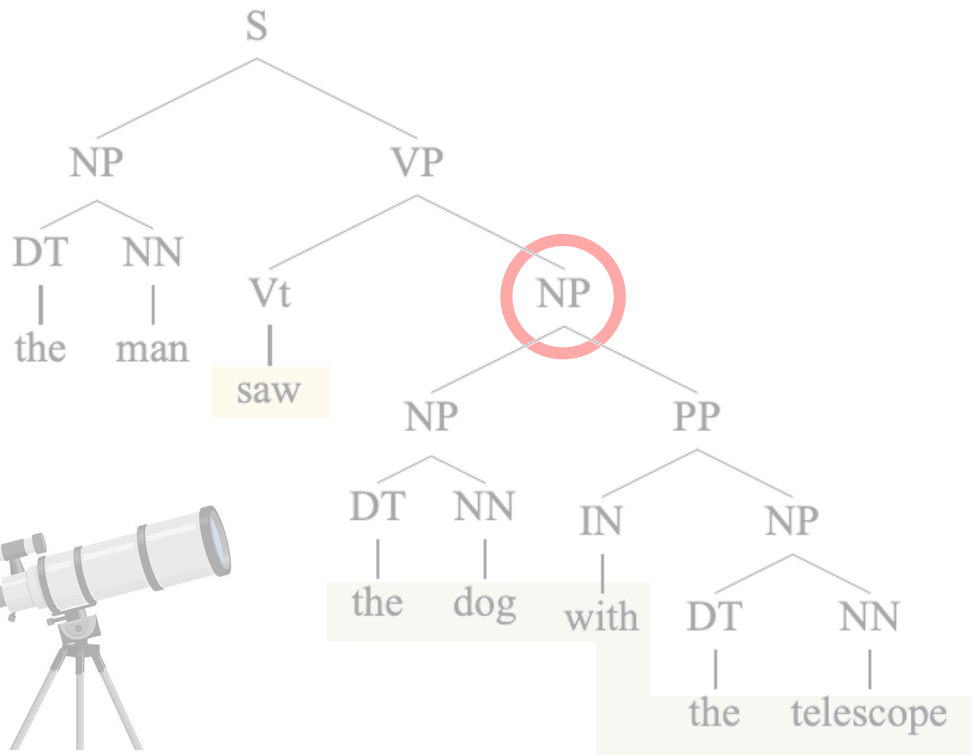
- " The man saw the dog with the telescope. "
- What do we understand here?

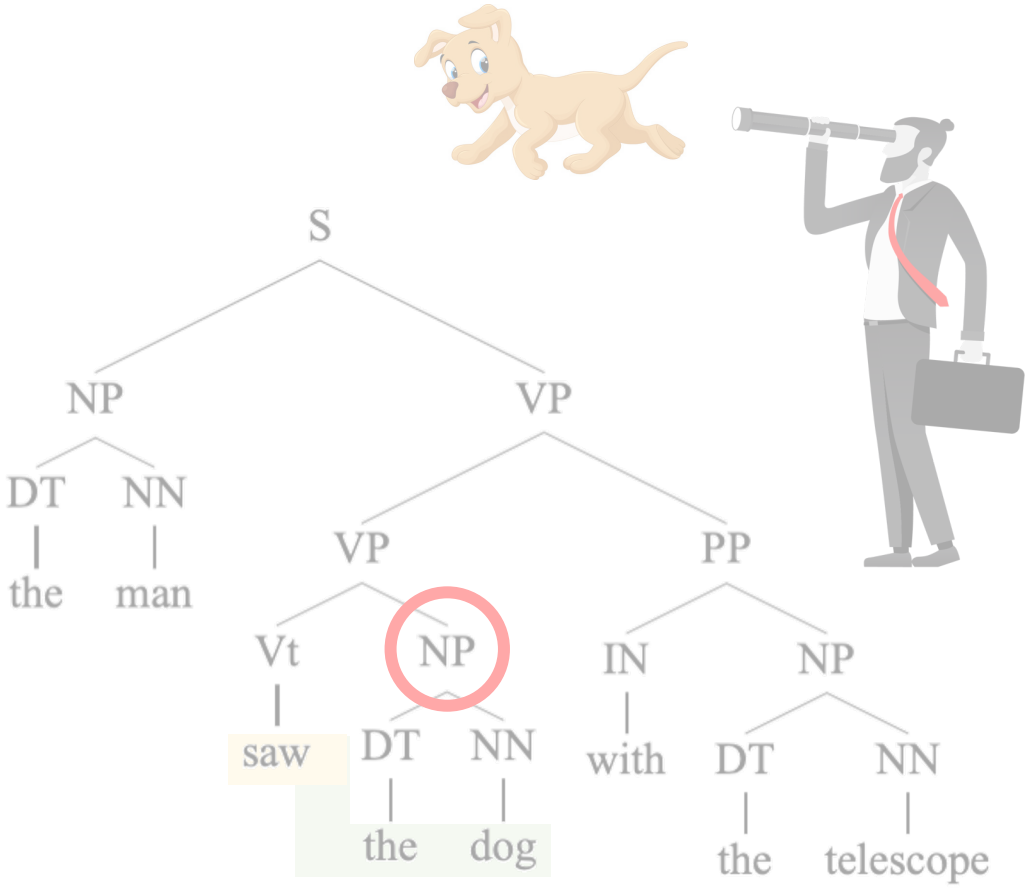




Key Challenge in Parsing: **Ambiguity**

- " The man saw the dog with the telescope. "
- What do we understand here?





ONE IS MORE PROBABLE THAN THE OTHER!

Lexicalised Parsing

Lexicalisation of PCFGs

- So far we have only considered the **probabilities of POS-based rules**, e.g.: P(JJ NP) → 0.3

- We can "lexicalise" that, by considering the probabilities of words occurring in different constituents.
  E.g.:
  - "money" → noun (most common, P=0.9) or adjective (P=0.1).
  - "money laundering" → can be NP NP or JJ NP.
    - However, "money" more likely to occur as a NP, as thus as part of NP NP rather than in JJ NP → then it increases P(NP NP).

Lexicalisation of PCFGs

- Probability of different verbal complement frames (i.e., "**subcategorisations**") depends on the verb:

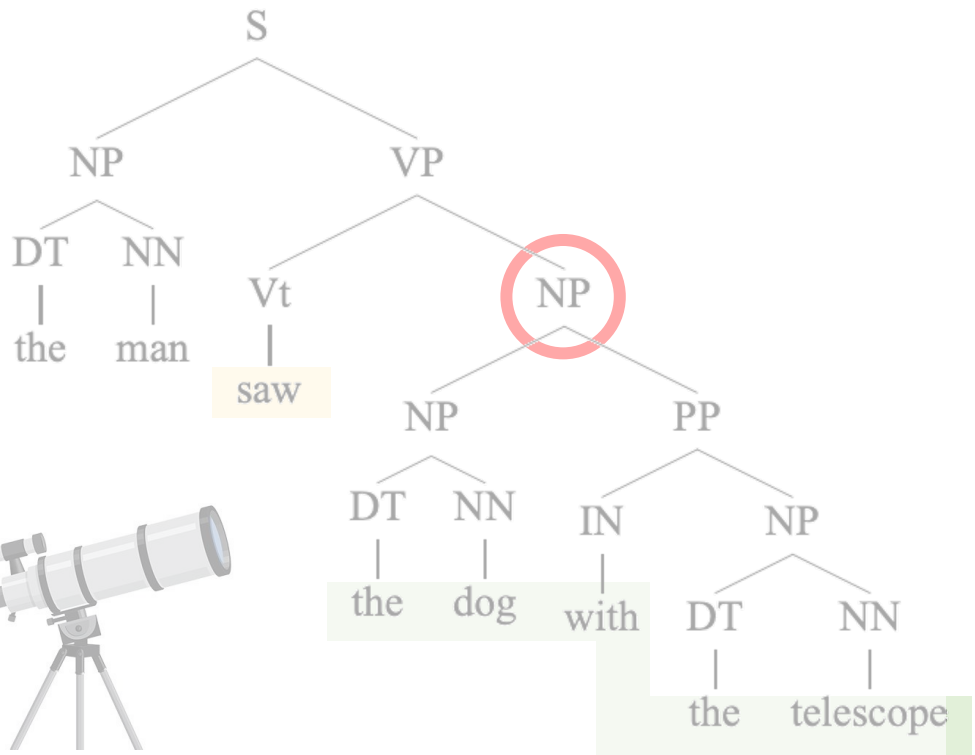
Local Tree	*come*	*take*	*think*	*want*
VP → V	9.5%	2.6%	4.6%	5.7%
VP → V NP	1.1%	32.1%	0.2%	13.9%
VP → V PP	34.5%	3.1%	7.1%	0.3%
VP → V SBAR	6.6%	0.3%	73.0%	0.2%
VP → V S	2.2%	1.3%	4.8%	70.8%
VP → V NP S	0.1%	5.7%	0.0%	0.3%
VP → V PRT NP	0.3%	5.8%	0.0%	0.0%
VP → V PRT PP	6.1%	1.5%	0.2%	0.0%

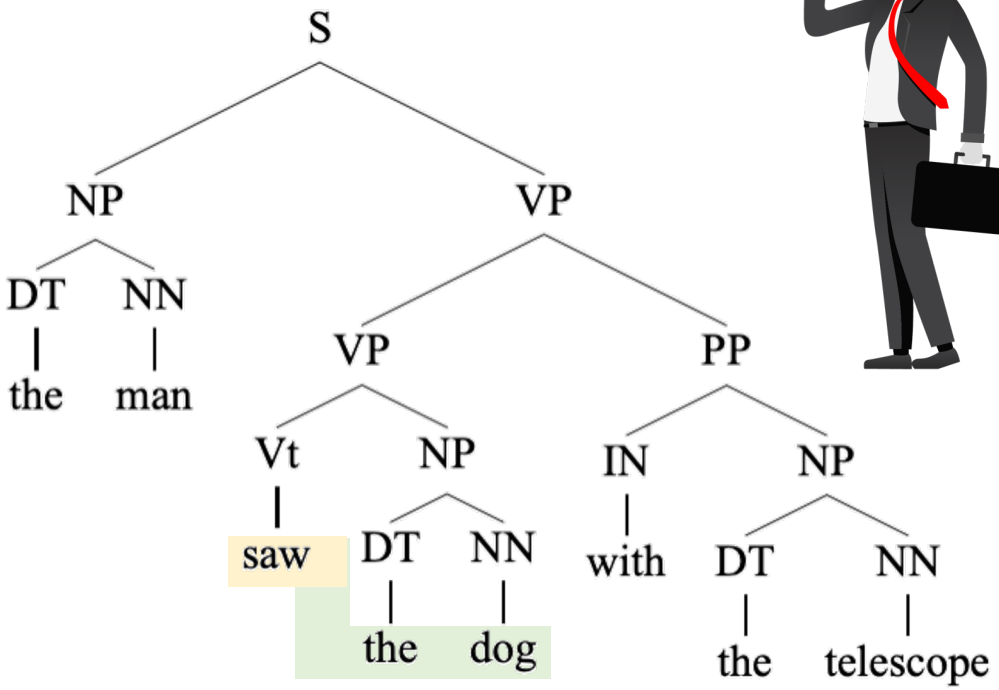
Example of PCFGs

Grammar		Lexicon
$S \rightarrow NP\ VP$	[.80]	$Det \rightarrow that$ [.10] | a [.30] | the [.60]
$S \rightarrow Aux\ NP\ VP$	[.15]	$Noun \rightarrow book$ [.10] | $flight$ [.30]
$S \rightarrow VP$	[.05]	| $meal$ [.015] | $money$ [.05]
$NP \rightarrow Pronoun$	[.35]	| $flight$ [.40] | $dinner$ [.10]
$NP \rightarrow Proper\text{-}Noun$	[.30]	$Verb \rightarrow book$ [.30] | $include$ [.30]
$NP \rightarrow Det\ Nominal$	[.20]	| $prefer$ [.40]
$NP \rightarrow Nominal$	[.15]	$Pronoun \rightarrow I$ [.40] | she [.05]
$Nominal \rightarrow Noun$	[.75]	| me [.15] | you [.40]
$Nominal \rightarrow Nominal\ Noun$	[.20]	$Proper\text{-}Noun \rightarrow Houston$ [.60]
$Nominal \rightarrow Nominal\ PP$	[.05]	| NWA [.40]
$VP \rightarrow Verb$	[.35]	$Aux \rightarrow does$ [.60] | can [40]
$VP \rightarrow Verb\ NP$	[.20]	$Preposition \rightarrow from$ [.30] | to [.30]
$VP \rightarrow Verb\ NP\ PP$	[.10]	| on [.20] | $near$ [.15]
$VP \rightarrow Verb\ PP$	[.15]	| $through$ [.05]
$VP \rightarrow Verb\ NP\ NP$	[.05]	
$VP \rightarrow VP\ PP$	[.15]	
$PP \rightarrow Preposition\ NP$	[1.0]	

Key Challenge in Parsing: **Ambiguity**

- " The man saw the dog with the telescope. "
- Lexicalised PCFGs can deal better with ambiguities.





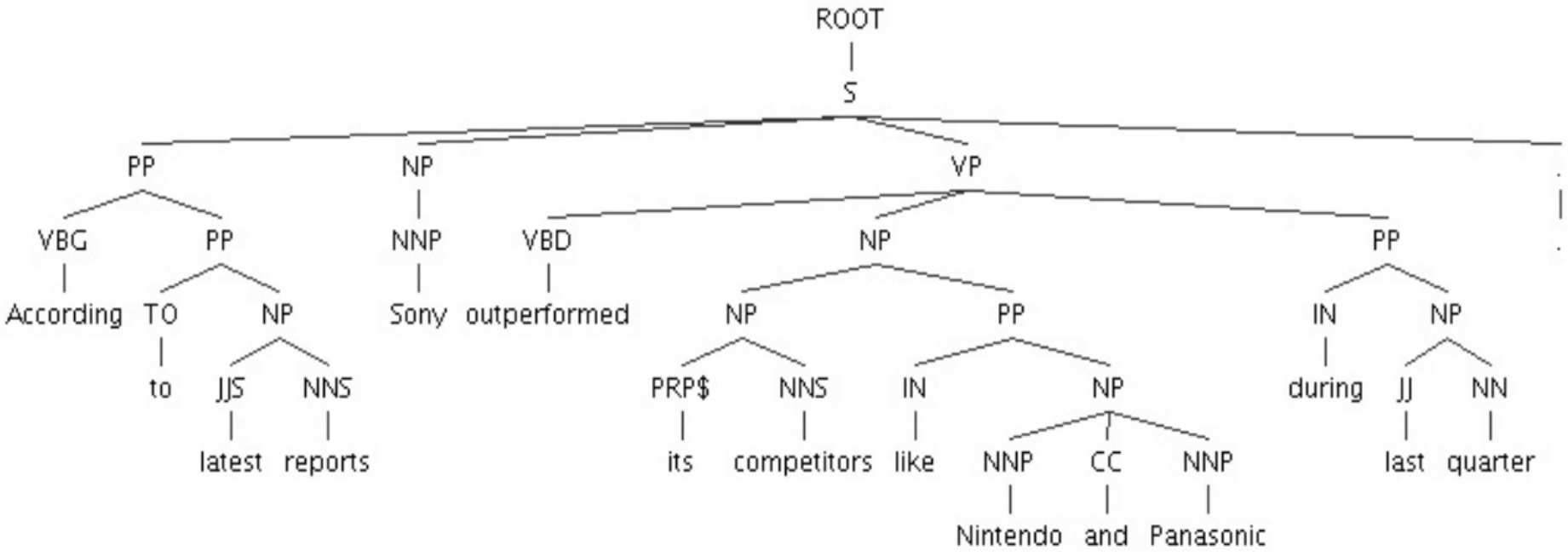
P(saw the dog)

>>

P(the dog with the telescope)

Constituent Tree

- Trees can be more complex.
  - E.g.: NPs in ***subject position*** tend to be pronouns, whereas NPs in ***object position*** tend to have full lexical form → **Split the pre-terminal** part-of-speech nodes
- CKY: break it down, bottom-up; POS tagging for leaves, then go up.



According to latest reports Sony outperformed its competitors like Nintendo and Panasonic during last quarter.

Penn Treebank Non-Terminals

S	Sentence or clause.
SBAR	Clause introduced by a (possibly empty) subordinating conjunction.
SBARQ	Direct question introduced by a *wh*-word or *wh*-phrase.
SINV	Inverted declarative sentence.
SQ	Inverted yes/no question, or main clause of a *wh*-question.
ADJP	Adjective Phrase.
ADVP	Adverb Phrase.
CONJP	Conjunction Phrase.
FRAG	Fragment.
INTJ	Interjection.
LST	List marker. Includes surrounding punctuation.
NAC	Not A Constituent; used within an NP.
NP	Noun Phrase.
NX	Used within certain complex NPs to mark the head.
PP	Prepositional Phrase.

-ADV	Constituent is used adverbially
-LOC	Constituent indicates event location
-PRD	Constituent serves as a sentence's predicate but is not a VP
-NOM	Constituent is used as a noun, e.g., *what I really like* is chocolate
-TMP	Constituent indicates when, how often, how long

CC	Coordinating conjunction
CD	Cardinal number
DT	Determiner
EX	Existential *there*
FW	Foreign word
IN	Preposition or subordinating conjunction
JJ	Adjective
JJR	Adjective, comparative
JJS	Adjective, superlative
LS	List item marker
MD	Modal
NN	Noun, singular or mass
NNS	Noun, plural
NNP	Proper noun, singular
NNPS	Proper noun, plural
PDT	Predeterminer

Penn Treebank Non-Terminals

PRN	Parenthetical.
PRT	Particle.
QP	Quantity Phrase (i.e., complex measure/amount) within NP.
RRC	Reduced Relative Clause.
UCP	Unlike Coordinated Phrase.
VP	Verb Phrase.
WHADJP	*Wh*-adjective Phrase, as in *how hot*.
WHADVP	*Wh*-adverb Phrase.
WHNP	*Wh*-noun Phrase, e.g. *who*, *which book*, *whose daughter*, *none of which*, or *how many leopards*.
WHPP	*Wh*-prepositional Phrase, e.g., *of which* or *by whose authority*.
X	Unknown, uncertain, or unbracketable.

POS	Possessive ending
PRP	Personal pronoun
PP$	Possessive pronoun
RB	Adverb
RBR	Adverb, comparative
RBS	Adverb, superlative
RP	Particle
SYM	Symbol (mathematical)
TO	The word *to*
UH	Interjection
VB	Verb, stem
VBD	Verb, past tense
VBG	Verb, present participle
VBN	Verb, past participle
VBP	Verb, present but not VBZ
VBZ	Verb, present, 3rd-person sing.
WDT	*wh*-determiner
WP	*wh*-pronoun
WP$	Possessive *wh*-pronoun
WRB	*wh*-adverb

State-of-the-Art

- PCFG achieves ~73% on Penn TreeBank
- State-of-the art ~92%: Lexicalised PCFG

Two types of syntactical structures

1. **Constituency** structures
   - Phrase Structure Grammar
   - Context-Free Grammars (CFGs) → and Lexicalised Grammars

Cocke-Kasami-Younger (CKY) algorithm

2. **Dependency** structures
   - Graph Algorithms
   - Transition-based Algorithms

Two types of syntactical structures

1. **Constituency** structures
   - Phrase Structure Grammar
   - Context-Free Grammars (CFGs) → and Lexicalised Grammars

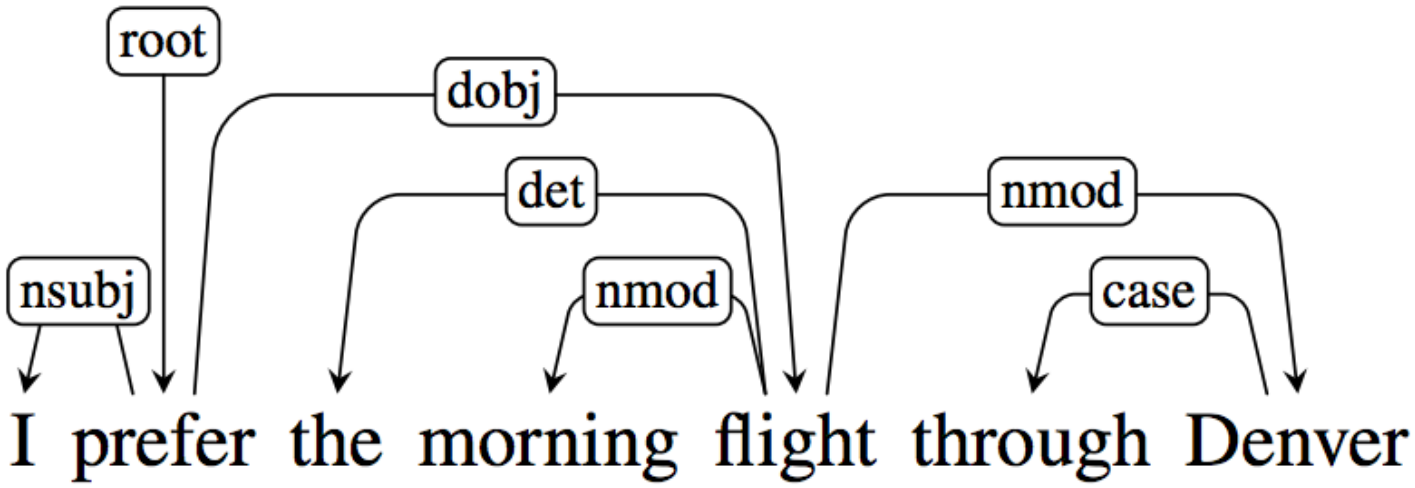
Cocke-Kasami-Younger (CKY) algorithm

2. **Dependency** structures
   - Graph Algorithms
   - Transition-based Algorithms

Dependency Parsing

Dependency Parsing

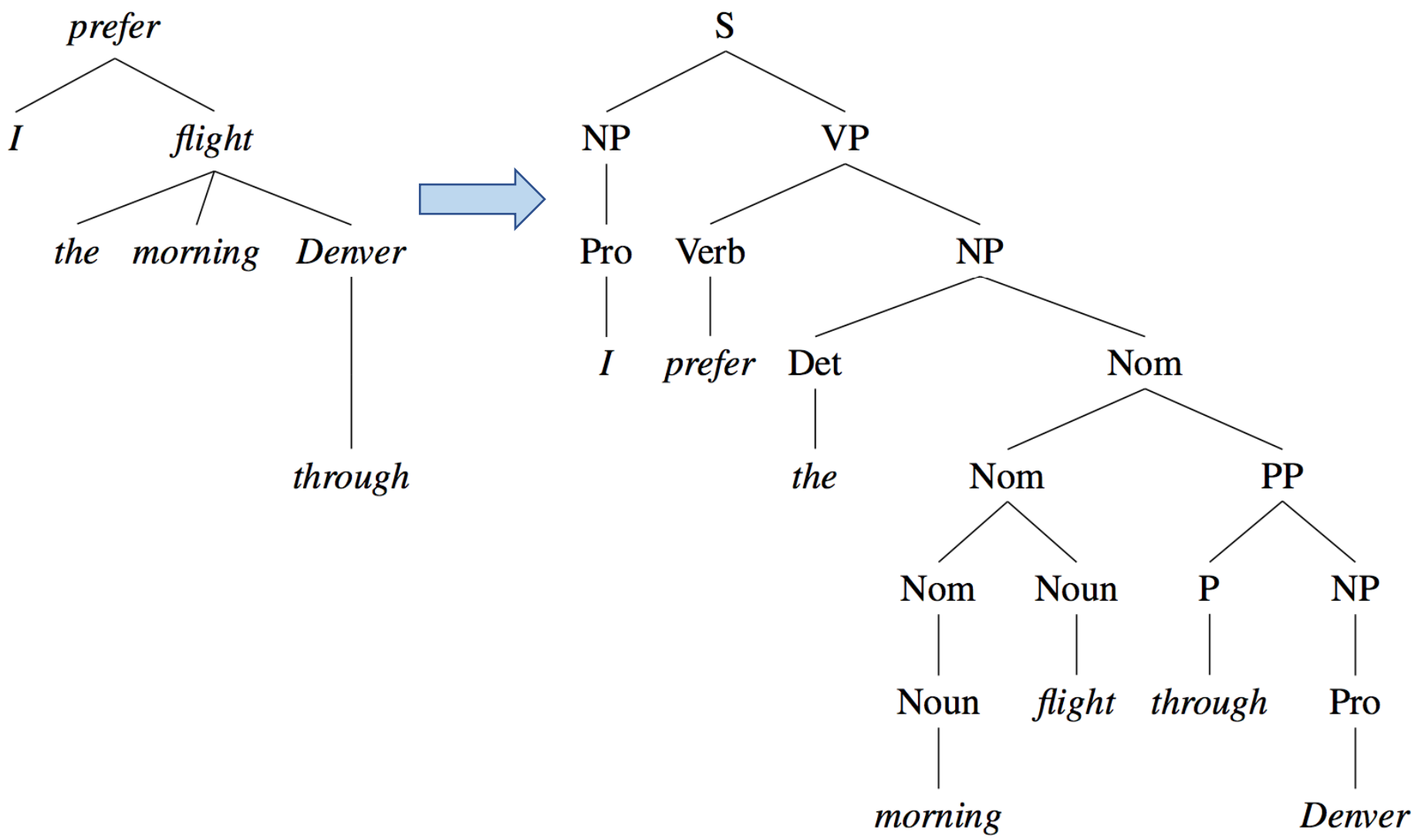
- Dependency syntax postulates that syntactic structure consists of lexical items linked by binary asymmetric relations ("arrows") called **dependencies**.
  - The arrow connects a **head** (governor, superior, regent) with a **dependent** (modifier, inferior, subordinate)



Dependency Relations from the Universal Dependency Set

Clausal Argument Relations	Description
NSUBJ	Nominal subject
DOBJ	Direct object
IOBJ	Indirect object
CCOMP	Clausal complement
XCOMP	Open clausal complement
Nominal Modifier Relations	**Description**
NMOD	Nominal modifier
AMOD	Adjectival modifier
NUMMOD	Numeric modifier
APPOS	Appositional modifier
DET	Determiner
CASE	Prepositions, postpositions and other case markers
Other Notable Relations	**Description**
CONJ	Conjunct
CC	Coordinating conjunction

Dependency Parsing



Dependency Parsing

- How do we decide which one's the **head**?
- Usually define heads in PCFG, e.g.:
  - S → <u>NP</u> VP
  - VP → <u>VBD</u> NP PP
  - NP → DT JJ <u>NN</u>

Dependency Parsing

- **Graph Algorithms:**
  1. Consider all word pairs
  2. Create a Maximum Spanning Tree for a sentence
- **Transition-based Approaches:**
  - Similar to how we parse a program
  - Shift-Reduce Parser: MaltParser

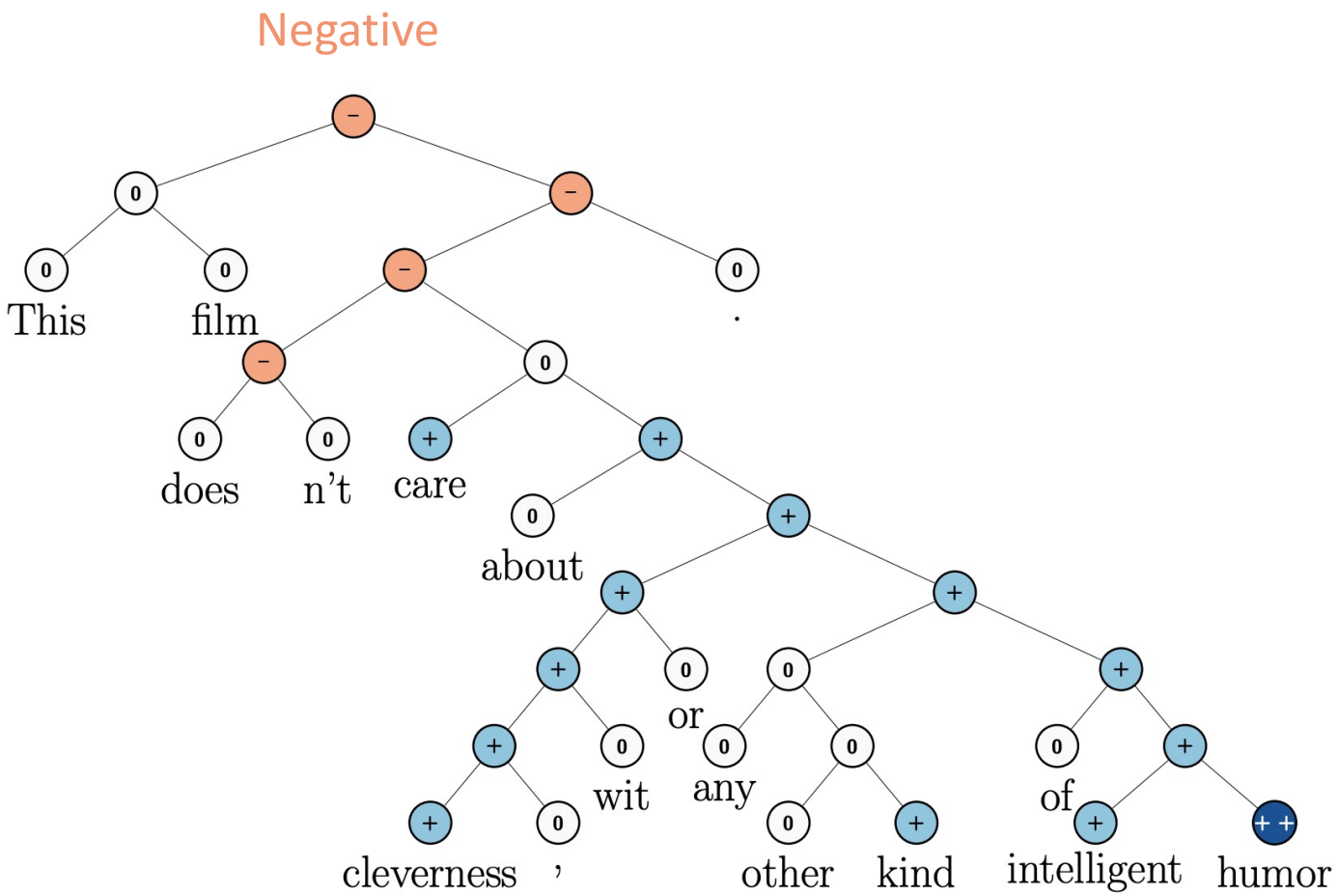
Resources

- There is a long list of Treebanks available for a wide range of languages, a good list can be found here:

  https://en.wikipedia.org/wiki/Treebank

Nested Sentiment Analysis

"This film doesn't care about cleverness, wit or any other kind of intelligent humor"



Recursive deep models for semantic compositionality over a sentiment treebank 13 Socher et al, EMNLP 2013

Grammar and Meaning

"Colorless Green Ideas Sleep Furiously"

- A **grammatically correct** sentence can be **semantically nonsensical.**
- This proves that **grammar** and **meaning are separate**
  - Order allows meaning to exist, but meaning is not just syntax.

Grammar and Meaning

"Colorless Green Ideas Sleep Furiously"

- A **grammatically correct** sentence can be **semantically nonsensical.**
- This proves that **grammar** and **meaning are separate**
  - **Order allows meaning to exist**, but meaning is not just syntax.

Reading

- Jurafsky, Daniel and James H. Martin. 2026. Speech and Language Processing. 3rd edition. **Chapters 18-19.**